

编译原理期中大作业实验报告

类 Rust 语言词法分析与语法分析器设计与实现

学院： 计算机科学与技术学院

专业： 计算机科学与技术

学号： 2351579 / 2253797 / 2352628

姓名： 程浩然 / 黄艺鑫 / 韦畅

指导教师： 高珍、卫志华

学期： 2026 年春季学期

2026 年 5 月 18 日

目录

摘要	5
1 概述	6
1.1 项目背景与目标	6
1.2 目标语言简介	6
1.3 团队分工与开发环境	7
1.4 系统架构	8
2 目标语言文法定义	9
2.1 词法规则	9
2.1.1 关键字	9
2.1.2 运算符	9
2.1.3 界符	10
2.1.4 常量类型	10
2.1.5 标识符与类型声明符	11
2.1.6 注释	11
2.2 语法规则	11
2.2.1 基础定义（规则 0.1 – 0.3）	11
2.2.2 程序结构（规则 1.1 – 1.5）	12
2.2.3 变量声明与赋值（规则 2.0 – 2.3）	13
2.2.4 表达式（规则 3.1 – 3.5）	14
2.2.5 选择结构（规则 4.1 – 4.3）	16
2.2.6 循环结构（规则 5.0 – 5.4）	17
2.2.7 引用与借用（规则 6.1 – 6.4）	18
2.2.8 函数表达式块（规则 7.0 – 7.4）	18
2.2.9 数组（规则 8.1 – 8.3）	19
2.2.10 元组（规则 9.1 – 9.3）	20

目录	2
3 词法分析器设计与实现	22
3.1 词法分析理论基础	22
3.2 Flex 工具介绍	23
3.3 Token 设计	24
3.3.1 Type 枚举定义	24
3.3.2 Token 类定义	24
3.4 正则表达式规则设计	25
3.4.1 常量识别规则	25
3.4.2 关键字识别规则	26
3.4.3 关键字与标识符的区分机制	27
3.4.4 运算符与界符识别	28
3.5 输出格式设计	29
3.6 TokenStream 接口	30
4 语法分析器设计与实现	32
4.1 语法分析理论基础	32
4.1.1 递归下降分析法原理	32
4.1.2 LL(1) 文法判定	33
4.1.3 左递归消除方法	33
4.2 语法树数据结构	33
4.2.1 Node 类定义	34
4.2.2 辅助函数	34
4.3 Parser 类实现	35
4.3.1 核心解析函数示例	36
4.4 表达式解析与运算符优先级	38
4.4.1 表达式解析函数	38
5 GUI 可视化模块	41
5.1 ImGui 框架简介	41

目录	3
5.2 界面布局设计	42
5.3 核心组件实现	42
5.3.1 Token 列表展示	42
5.3.2 语法树渲染	43
5.4 GUI 架构	44
6 构建与跨平台支持	45
6.1 CMake 构建系统	45
6.2 Linux/Windows 交叉编译配置	45
6.3 运行脚本说明	46
7 测试与结果分析	48
7.1 测试用例设计	48
7.2 词法分析输出示例	50
7.3 语法树输出示例	50
7.4 分规则测试用例与结果	51
7.4.1 程序结构测试（规则 1.1 – 1.5）	52
7.4.2 变量声明与赋值测试（规则 2.0 – 2.3）	52
7.4.3 表达式测试（规则 3.1 – 3.5）	53
7.4.4 选择结构测试（规则 4.1 – 4.3）	54
7.4.5 循环结构测试（规则 5.0 – 5.4）	55
7.4.6 引用与借用测试（规则 6.1 – 6.4）	55
7.4.7 数组测试（规则 8.1 – 8.3）	56
7.4.8 元组类型测试（规则 9.1）	57
7.5 运行效果截图	57
8 思考与分析	59
8.1 思考题一：1.4 函数输入中的形参列表可以识别怎样的语言？	59
8.2 思考题二：9.1 元组比 8.1 数组多了几条产生式，为什么？	60
8.3 部分羁绊分析	61

8.3.1	羁绊一：赋值语句 + 类型规则——类型不匹配应报错	61
8.3.2	羁绊二：for 循环 + 数组——仅数组为可迭代结构	61
8.3.3	羁绊三：赋值语句 + 变量不可变属性——对不可变变量赋值应报错	62
8.4	LR(1) 冲突分析	62
8.4.1	经典悬空 else 问题	62
8.4.2	规则 7.0 中的移进-归约冲突	63
8.4.3	其他潜在冲突分析	64
9	总结与反思	65
9.1	项目成果总结	65
9.2	遇到的问题与解决方案	65
9.3	知识收获与能力提升	66
9.4	改进方向与展望	66
9.5	个人想法与体会	67
9.5.1	对文法设计的见解	67
9.5.2	对实现方案的取舍思考	67
9.5.3	对 Rust 语言特性的分析	68
10	参考文献	69
附录 A	完整文法规则列表	70
A.1	完整产生式列表	70
附录 B	完整测试输入及输出	72
B.1	完整词法分析输出	72
B.2	完整语法树输出	72
B.3	运行命令示例	72

摘要

本项目实现了一个针对类 Rust 语言的前端编译器，包含完整的词法分析器和语法分析器模块。词法分析器使用 Flex 工具实现，能够识别关键字、运算符、常量、标识符等多种词法单元；语法分析器采用递归下降分析法，能够解析函数定义、变量声明、控制流语句和各类表达式结构。此外，项目还基于 ImGui 框架实现了图形化界面，支持词法单元流和语法树的可视化展示。

本报告详细阐述了目标语言的文法定义、词法分析与语法分析的理论基础、各模块的具体设计与实现方案，并通过完整的测试用例验证了系统的正确性。项目采用 CMake 构建系统，支持 Linux 和 Windows 双平台编译运行。

关键词：词法分析、语法分析、递归下降、Flex、编译原理、类 Rust 语言

1 概述

1.1 项目背景与目标

编译原理是计算机科学的核心课程之一，它系统性地介绍了从源代码到目标代码的完整转换过程。本次期中大作业旨在通过实践加深对编译器前端——词法分析与语法分析的理解，掌握相关理论与实现技术。

本项目的核心目标是：

1. 设计并实现一个完整的词法分析器，能够将源代码字符串转换为词法单元序列
2. 设计并实现一个语法分析器，能够将词法单元序列转换为语法树
3. 理解并应用词法分析（有限状态自动机）和语法分析（递归下降）的理论基础
4. 实现图形化界面，提供直观的分析结果展示

1.2 目标语言简介

本项目选择类 Rust 语言作为目标语言，主要基于以下考量：

1. **语法简洁：**Rust 语言语法结构清晰，既保留了现代语言特性，又避免了过于复杂的语法规则
2. **类型明确：**Rust 强调显式类型声明，便于语法分析时处理类型信息
3. **学习价值：**Rust 作为新兴编程语言，了解其语法特点对未来技术发展有参考意义

本项目支持的类 Rust 语言特性包括：

- 函数定义与调用
- 变量声明（支持 `mut` 可变修饰符）
- 基础数据类型（`i32`、`bool` 等）
- 数组类型与数组操作
- 控制流语句（`if-else`、`while`、`for-in`、`loop`）
- 表达式（算术运算、比较运算、范围表达式）
- 注释（行注释、块注释）

1.3 团队分工与开发环境

表 1: 团队分工表

姓名	学号	主要分工
程浩然	2351579	词法分析器设计与实现（Flex 规则编写、Token 体系设计）、词法-语法接口对接、GUI 可视化模块、跨平台构建系统
黄艺鑫	2253797	语法分析器设计与实现（递归下降解析器）、语法树数据结构设计、表达式优先级处理、CLI 命令行接口
韦畅	2352628	实验报告撰写、测试用例设计与验证、文法规则整理与分析

表 2: 开发环境配置

组件	版本/说明
操作系统	Linux (Ubuntu) / Windows 11
编译器	GCC/G++ (Linux), MinGW-w64 (Windows)
Flex 版本	2.6.4
CMake 版本	3.10+
GUI 框架	ImGui + GLFW + OpenGL3
构建工具链	vcpkg（依赖管理）

1.4 系统架构

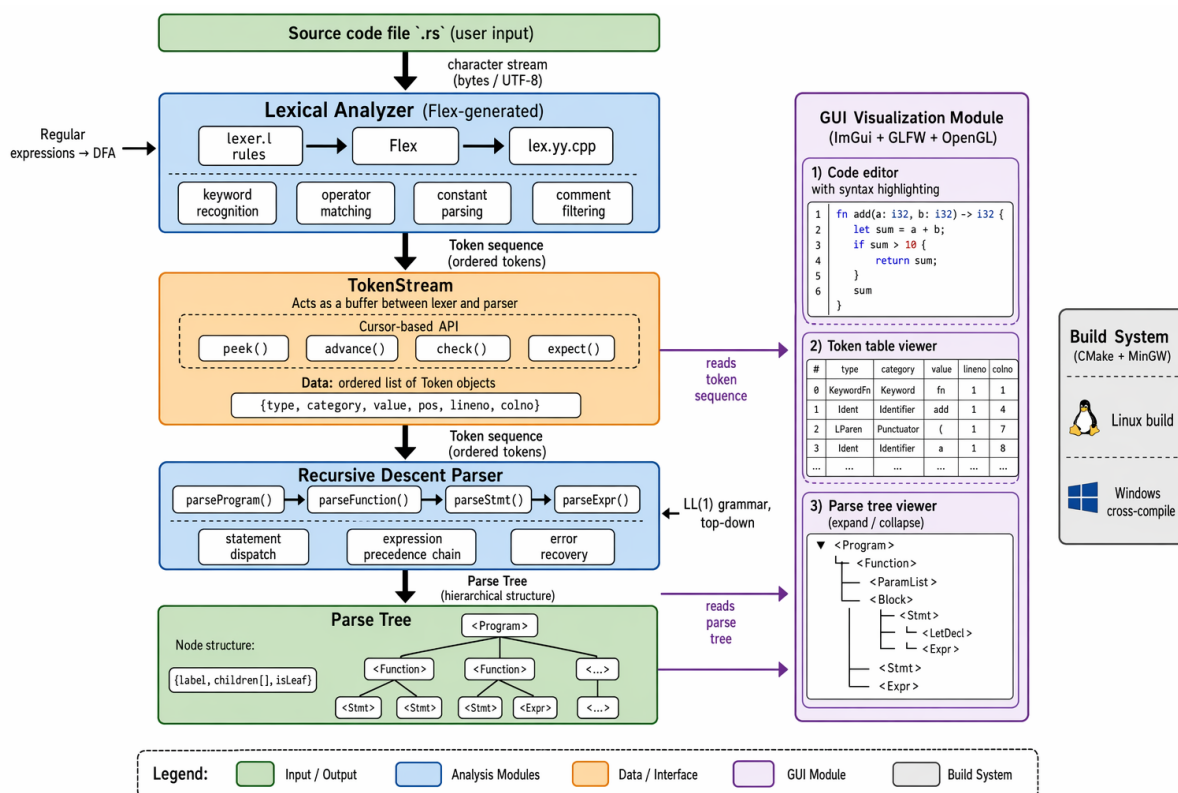


图 1: 系统总体架构图

系统由以下核心模块组成:

1. **词法分析器**: 负责将源代码字符串分解为词法单元 (Token)
2. **语法分析器**: 负责将 Token 序列组织为语法树
3. **TokenStream**: 作为词法分析器与语法分析器之间的接口
4. **GUI 模块**: 提供图形化界面, 展示分析结果
5. **构建系统**: 管理编译过程, 支持跨平台构建

2 目标语言文法定义

文法是描述语言语法结构的形式化方法，是编译器设计的基础。本章详细定义目标语言的词法规则和语法规则。

2.1 词法规则

词法规则定义了源代码中可以被识别的最小单位——词法单元（Token）。本项目将Token 分为以下几类：

2.1.1 关键字

关键字是语言中具有特殊含义的保留词，不能作为标识符使用。

表 3: 关键字定义表

关键字	种别码	功能说明
let	Let	变量声明
mut	Mut	可变修饰符
fn	Fn	函数定义
if	If	条件分支
else	Else	否则分支
while	While	当型循环
for	For	遍历循环
in	In	范围迭代
loop	Loop	无限循环
break	Break	跳出循环
continue	Continue	继续循环
return	Return	函数返回

2.1.2 运算符

运算符用于执行各种计算和比较操作。

表 4: 运算符定义表

运算符	种别码	功能说明
->	Arrow	返回类型声明

运算符	种别码	功能说明
..	DotDot	范围运算符
.	Dot	成员访问
==	EqualEqual	相等比较
!=	NotEqual	不等比较
>=	GreaterEqual	大于等于
<=	LessEqual	小于等于
>	Greater	大于
<	Less	小于
+	Plus	加法
-	Minus	减法
*	Star	乘法/解引用
/	Slash	除法
&	Ampersand	引用/位运算

2.1.3 界符

界符用于分隔和组织语法结构。

表 5: 界符定义表

界符	种别码	功能说明
=	Assign	赋值
;	Semicolon	语句结束
:	Colon	类型声明
,	Comma	参数分隔
(	LParen	左圆括号
)	RParen	右圆括号
{	LBrace	左花括号
}	RBrace	右花括号
[	LBracket	左方括号
]	RBracket	右方括号
#	End	程序结束标记

2.1.4 常量类型

常量是程序中的固定值，本项目支持四种常量类型：

表 6: 常量类型定义

常量类型	种别码	示例
整数常量	IntegerConstant	42, 0xFF, 0b1010
浮点常量	FloatingConstant	3.14, 2.5e10
字符串常量	StringConst	"hello"
字符常量	CharacterConstant	'a', '\n'

2.1.5 标识符与类型声明符

表 7: 标识符与类型声明符

类型	种别码	说明
标识符	Identifier	以字母或下划线开头，后跟字母、数字或下划线
类型声明符	Declarator	如 i32

2.1.6 注释

表 8: 注释类型

注释类型	种别码	格式
行注释	LineComment	// 开头，至行尾
块注释	BlockComment	/* */ 包围

2.2 语法规则

本节按作业要求的编号体系（0.1–9.3）逐条列出文法产生式。作业明确要求完成的必选规则为 0.1–0.3、1.1–1.5、2.0–2.2、3.1–3.5、4.1、5.0–5.1，其余为扩展规则。每条规则标注实现状态：**[已实现]** 或 **[未实现]**，必选规则额外注明”必选”。

2.2.1 基础定义（规则 0.1 – 0.3）

规则 0.1 变量属性

[已实现 · 必选]

$\langle \text{变量属性} \rangle \rightarrow \text{mut} \quad (1)$

变量属性用于修饰可变变量。实现中通过 `ts.check("Mut")` 判断是否存在 `mut` 关键字。

规则 0.2 类型

[已实现 · 必选]

$$\langle \text{类型} \rangle \rightarrow \text{i32} \quad (2)$$

基础类型仅包含 i32，在词法分析器中以种别码 I32（类型为 declerator）识别。后续扩展规则将增加数组类型、引用类型、元组类型等形式。

规则 0.3 左值

[已实现 · 必选]

$$\langle \text{左值} \rangle \rightarrow \langle \text{可取引用} \rangle \quad (3)$$

$$\langle \text{可取引用} \rangle \rightarrow \langle \text{可取元素} \rangle \quad (4)$$

$$\langle \text{可取元素} \rangle \rightarrow \text{ID} \quad (5)$$

左值的处理采用统一策略：先调用 `parseExpr()` 解析表达式，再检查是否跟随赋值号 `=`，从而区分赋值语句与表达式语句。后续扩展规则为 $\langle \text{可取引用} \rangle$ 和 $\langle \text{可取元素} \rangle$ 添加更多候选产生式。

2.2.2 程序结构（规则 1.1 – 1.5）

规则 1.1 基础程序

[已实现 · 必选]

$$Program \rightarrow \langle \text{声明串} \rangle \quad (6)$$

$$\langle \text{声明串} \rangle \rightarrow \epsilon \mid \langle \text{声明} \rangle \langle \text{声明串} \rangle \quad (7)$$

$$\langle \text{声明} \rangle \rightarrow \langle \text{函数声明} \rangle \quad (8)$$

$$\langle \text{函数声明} \rangle \rightarrow \langle \text{函数头声明} \rangle \langle \text{语句块} \rangle \quad (9)$$

$$\langle \text{函数头声明} \rangle \rightarrow \text{fn ID} (\langle \text{形参列表} \rangle) \quad (10)$$

$$\langle \text{形参列表} \rangle \rightarrow \epsilon \quad (11)$$

$$\langle \text{语句块} \rangle \rightarrow \{ \langle \text{语句串} \rangle \} \quad (12)$$

$$\langle \text{语句串} \rangle \rightarrow \epsilon \quad (13)$$

规则 1.2 语句（拓展 1.1）

[已实现 · 必选]

$$\langle \text{语句串} \rangle \rightarrow \langle \text{语句} \rangle \langle \text{语句串} \rangle \quad (14)$$

$$\langle \text{语句} \rangle \rightarrow ; \quad (15)$$

在 `parseStmt()` 中, 若当前 Token 为 Semicolon, 则创建 EmptyStmt 节点。

规则 1.3 返回语句 (拓展 1.2) [已实现 · 必选]

⟨语句⟩ → ⟨返回语句⟩ (16)

⟨返回语句⟩ → `return` ; (17)

在 `parseReturnStmt()` 中, 若 `return` 后直接是分号, 则不解析表达式。

规则 1.4 函数输入 (依赖 0.1、0.2, 拓展 1.1) [已实现 · 必选]

⟨形参列表⟩ → ⟨形参⟩ | ⟨形参⟩ , ⟨形参列表⟩ (18)

⟨形参⟩ → ⟨变量属性⟩ ID : ⟨类型⟩ (19)

由 `parseParamList()` 和 `parseParam()` 实现。若当前符号为 RParen 则选择 ϵ , 否则解析形参并以逗号为分隔循环解析。

规则 1.5 函数输出 (依赖 0.2、3.1, 拓展 1.1、1.3) [已实现 · 必选]

⟨函数头声明⟩ → `fn` ID (⟨形参列表⟩) -> ⟨类型⟩ (20)

⟨返回语句⟩ → `return` ⟨表达式⟩ ; (21)

在 `parseFuncHeader()` 中, 若 RParen 后检测到 Arrow (`->`) 则继续解析返回类型。

2.2.3 变量声明与赋值 (规则 2.0 – 2.3)

规则 2.0 变量声明 (依赖 0.1、0.2) [已实现 · 必选]

⟨变量声明⟩ → ⟨变量属性⟩ ID (22)

⟨变量声明⟩ → ⟨变量属性⟩ ID : ⟨类型⟩ (23)

在 `parseLetStmt()` 中创建 VarDecl 子节点, 可选地解析 `mut`、标识符和类型注解。

规则 2.1 变量声明语句 (依赖 2.0, 拓展 1.2) [已实现 · 必选]

⟨语句⟩ → ⟨变量声明语句⟩ (24)

⟨变量声明语句⟩ → `let` ⟨变量声明⟩ ; (25)

规则 2.2 赋值语句（依赖 0.3、3.1，拓展 1.2）**[已实现 · 必选]**

$$\langle \text{语句} \rangle \rightarrow \langle \text{赋值语句} \rangle \quad (26)$$

$$\langle \text{赋值语句} \rangle \rightarrow \langle \text{左值} \rangle = \langle \text{表达式} \rangle ; \quad (27)$$

由 `parseExprOrAssignStmt()` 实现：先解析表达式作为左部，若后跟 `=` 则构造赋值语句节点，否则构造表达式语句节点。

规则 2.3 变量声明赋值语句（依赖 2.0、3.1，拓展 1.2）**[已实现 · 扩展]**

$$\langle \text{语句} \rangle \rightarrow \langle \text{变量声明赋值语句} \rangle \quad (28)$$

$$\langle \text{变量声明赋值语句} \rangle \rightarrow \text{let } \langle \text{变量声明} \rangle = \langle \text{表达式} \rangle ; \quad (29)$$

在 `parseLetStmt()` 中，变量声明后若检测到 `Assign (=)` 则继续解析初始化表达式。

2.2.4 表达式（规则 3.1 – 3.5）

表达式文法原始形式存在左递归，不利于递归下降分析。以下给出消除左递归后的等价形式。

规则 3.1 基本表达式（依赖 0.3，拓展 1.2）**[已实现 · 必选]**

$$\langle \text{语句} \rangle \rightarrow \langle \text{表达式} \rangle ; \quad (30)$$

$$\langle \text{表达式} \rangle \rightarrow \langle \text{加减表达式} \rangle \quad (31)$$

$$\langle \text{加减表达式} \rangle \rightarrow \langle \text{项} \rangle \quad (32)$$

$$\langle \text{项} \rangle \rightarrow \langle \text{因子} \rangle \quad (33)$$

$$\langle \text{因子} \rangle \rightarrow \text{NUM} \mid \langle \text{左值} \rangle \mid (\langle \text{表达式} \rangle) \quad (34)$$

规则 3.2 增加比较运算（依赖 3.1，拓展 3.1）**[已实现 · 必选]**

$$\langle \text{表达式} \rangle \rightarrow \langle \text{表达式} \rangle \langle \text{比较运算符} \rangle \langle \text{加减表达式} \rangle \quad (35)$$

$$\langle \text{比较运算符} \rangle \rightarrow < \mid <= \mid > \mid >= \mid == \mid != \quad (36)$$

由 `parseCmpExpr()` 实现，使用循环结构处理连续的比较运算。

规则 3.3 增加加减运算（依赖 3.1，拓展 3.1）

[已实现 · 必选]

$$\langle \text{加减表达式} \rangle \rightarrow \langle \text{加减表达式} \rangle \langle \text{加减运算符} \rangle \langle \text{项} \rangle \quad (37)$$

$$\langle \text{加减运算符} \rangle \rightarrow + \mid - \quad (38)$$

由 `parseAddExpr()` 实现。

规则 3.4 增加乘除运算（依赖 3.1，拓展 3.1）

[已实现 · 必选]

$$\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle \langle \text{乘除运算符} \rangle \langle \text{因子} \rangle \quad (39)$$

$$\langle \text{乘除运算符} \rangle \rightarrow * \mid / \quad (40)$$

由 `parseTerm()` 实现。

规则 3.5 增加函数调用（依赖 3.1，拓展 3.1）

[已实现 · 必选]

$$\langle \text{因子} \rangle \rightarrow \text{ID} (\langle \text{实参列表} \rangle) \quad (41)$$

$$\langle \text{实参列表} \rangle \rightarrow \epsilon \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle , \langle \text{实参列表} \rangle \quad (42)$$

在 `parseAtom()` 中，若标识符后跟 `LParen` 则构造 `CallExpr` 节点。参数列表由 `parseArgList()` 解析。

表达式消除左递归后的完整形式。上述 3.2–3.4 的原始文法含有左递归，实际实现中采用消除左递归后的等价文法：

$$Expr \rightarrow RangeExpr \quad (43)$$

$$RangeExpr \rightarrow CmpExpr \mid CmpExpr \dots CmpExpr \quad (44)$$

$$CmpExpr \rightarrow AddExpr \mid CmpExpr \ CmpOp \ AddExpr \quad (45)$$

$$CmpOp \rightarrow == \mid != \mid > \mid >= \mid < \mid <= \quad (46)$$

$$AddExpr \rightarrow MulExpr \mid AddExpr \ AddOp \ MulExpr \quad (47)$$

$$AddOp \rightarrow + \mid - \quad (48)$$

$$MulExpr \rightarrow Unary \mid MulExpr \ MulOp \ Unary \quad (49)$$

$$MulOp \rightarrow * \mid / \quad (50)$$

$$Unary \rightarrow \& \ [mut] \ Unary \mid * \ Unary \mid Atom \quad (51)$$

$$Atom \rightarrow Constant \mid (\ Expr \) \mid ArrayLit \quad (52)$$

$$\mid Identifier \ [CallExpr \mid IndexExpr] \quad (53)$$

$$CallExpr \rightarrow (\ ArgList \) \quad (54)$$

$$IndexExpr \rightarrow [\ Expr \] \quad (55)$$

$$ArgList \rightarrow \epsilon \mid Expr \mid Expr \ , \ ArgList \quad (56)$$

$$ArrayLit \rightarrow [\ ExprList \] \quad (57)$$

$$ExprList \rightarrow \epsilon \mid Expr \mid Expr \ , \ ExprList \quad (58)$$

运算符优先级从低到高为：范围 $\dots$ $\rightarrow$ 比较 $\rightarrow$ 加减 $\rightarrow$ 乘除 $\rightarrow$ 单目 $\rightarrow$ 原子，通过函数调用的嵌套层次保证高优先级运算符先被归约。

2.2.5 选择结构（规则 4.1 – 4.3）

规则 4.1 选择结构（依赖 1.1、3.1，拓展 1.2）

[已实现 · 必选]

$$\langle \text{语句} \rangle \rightarrow \langle \text{if 语句} \rangle \quad (59)$$

$$\langle \text{if 语句} \rangle \rightarrow \text{if } \langle \text{表达式} \rangle \langle \text{语句块} \rangle \langle \text{else 部分} \rangle \quad (60)$$

$$\langle \text{else 部分} \rangle \rightarrow \epsilon \quad (61)$$

规则 4.2 增加 else（依赖 1.1，拓展 4.1）

[已实现 · 扩展]

$$\langle \text{else 部分} \rangle \rightarrow \text{else } \langle \text{语句块} \rangle \quad (62)$$

规则 4.3 增加 `else if` (依赖 1.1, 拓展 4.1) [已实现 · 扩展]

$$\langle \text{else 部分} \rangle \rightarrow \text{else if } \langle \text{表达式} \rangle \langle \text{语句块} \rangle \langle \text{else 部分} \rangle \quad (63)$$

由 `parseIfStmt()` 实现, `else` 部分通过递归调用 `parseIfStmt()` 处理 `else if` 链。

2.2.6 循环结构 (规则 5.0 – 5.4)

规则 5.0 循环语句 (拓展 1.2) [已实现 · 必选]

$$\langle \text{语句} \rangle \rightarrow \langle \text{循环语句} \rangle \quad (64)$$

`parseStmt()` 根据 `While`、`For`、`Loop` 关键字分发到对应解析函数。

规则 5.1 `while` 循环 (依赖 1.1、3.1, 拓展 5.0) [已实现 · 必选]

$$\langle \text{循环语句} \rangle \rightarrow \langle \text{while 语句} \rangle \quad (65)$$

$$\langle \text{while 语句} \rangle \rightarrow \text{while } \langle \text{表达式} \rangle \langle \text{语句块} \rangle \quad (66)$$

规则 5.2 `for` 循环 (依赖 1.1、2.0、3.1, 拓展 5.0) [已实现 · 扩展]

$$\langle \text{循环语句} \rangle \rightarrow \langle \text{for 语句} \rangle \quad (67)$$

$$\langle \text{for 语句} \rangle \rightarrow \text{for } \langle \text{变量声明} \rangle \text{ in } \langle \text{可迭代结构} \rangle \langle \text{语句块} \rangle \quad (68)$$

$$\langle \text{可迭代结构} \rangle \rightarrow \langle \text{表达式} \rangle \dots \langle \text{表达式} \rangle \quad (69)$$

由 `parseForStmt()` 实现。范围表达式 `start..end` 在 `parseExpr()` 中作为 `RangeExpr` 节点处理。

规则 5.3 `loop` 循环 (依赖 1.1, 拓展 5.0) [已实现 · 扩展]

$$\langle \text{循环语句} \rangle \rightarrow \langle \text{loop 语句} \rangle \quad (70)$$

$$\langle \text{loop 语句} \rangle \rightarrow \text{loop } \langle \text{语句块} \rangle \quad (71)$$

规则 5.4 `break` 和 `continue` (拓展 1.2) [已实现 · 扩展]

$$\langle \text{语句} \rangle \rightarrow \text{break ;} \mid \text{continue ;} \quad (72)$$

2.2.7 引用与借用（规则 6.1 – 6.4）

规则 6.1 变量不可变属性（拓展 0.1） [已实现 · 扩展]

$$\langle \text{变量属性} \rangle \rightarrow \epsilon \quad (73)$$

变量属性可以为空，即 `let a;` 声明不可变变量。实现中 `mut` 为可选项，不出现时即对应此规则。

规则 6.2 不可变引用（依赖 0.2、0.3，拓展 0.2、0.3） [已实现 · 扩展]

$$\langle \text{类型} \rangle \rightarrow \& \langle \text{类型} \rangle \quad (74)$$

$$\langle \text{可取引用} \rangle \rightarrow \& \langle \text{可取引用} \rangle \quad (75)$$

在 `parseType()` 中，若当前符号为 `Ampersand(&)` 则解析引用类型。在 `parseUnary()` 中处理取地址表达式。

规则 6.3 可变引用（依赖 0.2、0.3，拓展 0.2、0.3） [已实现 · 扩展]

$$\langle \text{类型} \rangle \rightarrow \& \text{mut} \langle \text{类型} \rangle \quad (76)$$

$$\langle \text{可取引用} \rangle \rightarrow \& \text{mut} \langle \text{可取引用} \rangle \quad (77)$$

`parseType()` 在解析 `&` 后检查是否跟随 `Mut`。

规则 6.4 借用（解引用）（依赖 0.3，拓展 0.3） [已实现 · 扩展]

$$\langle \text{左值} \rangle \rightarrow * \langle \text{左值} \rangle \quad (78)$$

在 `parseUnary()` 中，若当前符号为 `Star(*)` 则构造 `DerefExpr` 节点。赋值语句 `*a = 3;` 由 `parseExprOrAssignStmt()` 正确处理。

2.2.8 函数表达式块（规则 7.0 – 7.4）

规则 7.0 函数表达式语句块（依赖 1.1、3.1，拓展 3.1） [未实现]

$$\langle \text{函数表达式语句块} \rangle \rightarrow \{ \langle \text{函数表达式语句串} \rangle \} \quad (79)$$

$$\langle \text{函数表达式语句串} \rangle \rightarrow \langle \text{表达式} \rangle | \langle \text{语句} \rangle \langle \text{函数表达式语句串} \rangle \quad (80)$$

规则 7.1 函数表达式块作为表达式（依赖 7.0，拓展 0.3） [未实现]

$$\langle \text{可取元素} \rangle \rightarrow \langle \text{函数表达式语句块} \rangle \quad (81)$$

规则 7.2 函数表达式块作为函数体（依赖 1.1、7.0，拓展 1.1） [未实现]

$$\langle \text{函数声明} \rangle \rightarrow \langle \text{函数头声明} \rangle \langle \text{函数表达式语句块} \rangle \quad (82)$$

规则 7.3 选择表达式（依赖 3.1、7.0，拓展 0.3） [未实现]

$$\langle \text{可取元素} \rangle \rightarrow \langle \text{选择表达式} \rangle \quad (83)$$

$$\langle \text{选择表达式} \rangle \rightarrow \text{if } \langle \text{表达式} \rangle \langle \text{函数表达式语句块} \rangle \text{ else } \langle \text{函数表达式语句块} \rangle \quad (84)$$

规则 7.4 循环表达式（依赖 3.1、5.3，拓展 0.3） [未实现]

$$\langle \text{可取元素} \rangle \rightarrow \langle \text{loop 语句} \rangle \quad (85)$$

$$\langle \text{语句} \rangle \rightarrow \text{break } \langle \text{表达式} \rangle ; \quad (86)$$

规则 7.0–7.4 涉及**块表达式**（block expression）语义：花括号内最后一个不带分号的表达式作为整个块的返回值。这要求在语法分析阶段区分”语句”与”尾表达式”，实现复杂度较高，当前版本暂未实现，作为后续改进方向。

2.2.9 数组（规则 8.1 – 8.3）

规则 8.1 数组类型（依赖 0.2，拓展 0.2） [已实现 · 扩展]

$$\langle \text{类型} \rangle \rightarrow [\langle \text{类型} \rangle ; \text{NUM}] \quad (87)$$

在 `parseType()` 中，若当前符号为 `LBracket` 则解析数组类型 `[T; N]`。

规则 8.2 数组表达式（依赖 3.1，拓展 0.3、5.2） [已实现 · 扩展]

$$\langle \text{可取元素} \rangle \rightarrow [\langle \text{数组元素列表} \rangle] \quad (88)$$

$$\langle \text{数组元素列表} \rangle \rightarrow \epsilon \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle , \langle \text{数组元素列表} \rangle \quad (89)$$

$$\langle \text{可迭代结构} \rangle \rightarrow \langle \text{表达式} \rangle \quad (90)$$

在 `parseAtom()` 中，若当前符号为 `LBracket` 且上下文非类型声明，则解析数组字面量 `[1, 2, 3]`。

规则 8.3 数组元素访问（依赖 0.3、3.1，拓展 0.3） [已实现 · 扩展]

$$\langle \text{可取元素} \rangle \rightarrow \langle \text{可取元素} \rangle [\langle \text{表达式} \rangle] \quad (91)$$

在 `parseAtom()` 中，标识符后若跟 `LBracket` 则构造 `IndexExpr` 节点，如 `arr[0]`。

2.2.10 元组（规则 9.1 – 9.3）

规则 9.1 元组类型（依赖 0.2，拓展 0.2） [已实现 · 扩展]

$$\langle \text{类型} \rangle \rightarrow (\langle \text{元组类型内部} \rangle) \quad (92)$$

$$\langle \text{元组类型内部} \rangle \rightarrow \epsilon \mid \langle \text{类型} \rangle , \langle \text{类型列表} \rangle \quad (93)$$

$$\langle \text{类型列表} \rangle \rightarrow \epsilon \mid \langle \text{类型} \rangle \mid \langle \text{类型} \rangle , \langle \text{类型列表} \rangle \quad (94)$$

在 `parseType()` 中，若当前符号为 `LParen` 则解析元组类型，如 `(i32, i32)`。

规则 9.2 元组表达式（依赖 3.1，拓展 0.3） [未实现]

$$\langle \text{可取元素} \rangle \rightarrow (\langle \text{元组赋值内部} \rangle) \quad (95)$$

$$\langle \text{元组赋值内部} \rangle \rightarrow \epsilon \mid \langle \text{表达式} \rangle , \langle \text{元组元素列表} \rangle \quad (96)$$

$$\langle \text{元组元素列表} \rangle \rightarrow \epsilon \mid \langle \text{表达式} \rangle \mid \langle \text{表达式} \rangle , \langle \text{元组元素列表} \rangle \quad (97)$$

当前 `parseAtom()` 将 `(expr)` 解析为括号表达式，无法区分单元元组 `()`、单元元素元组 `(1,)` 和括号表达式 `(1)`，需要引入更精细的向前查看逻辑。

规则 9.3 元组元素访问（依赖 0.3，拓展 0.3） [未实现]

$$\langle \text{可取元素} \rangle \rightarrow \langle \text{可取元素} \rangle . \text{NUM} \quad (98)$$

如 `tuple.0`、`tuple.1`。需要在 `parseAtom()` 中为标识符添加点号后缀的解析逻辑。

实现状态汇总。下表汇总了所有规则的实现情况：

表 9: 语法规则实现状态汇总

编号	规则名称	属性	状态
0.1	变量属性	必选	✓
0.2	类型	必选	✓
0.3	左值	必选	✓
1.1	基础程序	必选	✓
1.2	语句	必选	✓
1.3	返回语句	必选	✓
1.4	函数输入	必选	✓
1.5	函数输出	必选	✓
2.0	变量声明	必选	✓
2.1	变量声明语句	必选	✓
2.2	赋值语句	必选	✓
2.3	变量声明赋值语句	扩展	✓
3.1	基本表达式	必选	✓
3.2	比较运算	必选	✓
3.3	加减运算	必选	✓
3.4	乘除运算	必选	✓
3.5	函数调用	必选	✓
4.1	if 语句	必选	✓
4.2	else	扩展	✓
4.3	else if	扩展	✓
5.0	循环语句	必选	✓
5.1	while 循环	必选	✓
5.2	for 循环	扩展	✓
5.3	loop 循环	扩展	✓
5.4	break/continue	扩展	✓
6.1	变量不可变属性	扩展	✓
6.2	不可变引用	扩展	✓
6.3	可变引用	扩展	✓
6.4	借用（解引用）	扩展	✓
7.0	函数表达式语句块	扩展	✗
7.1	函数表达式块作为表达式	扩展	✗
7.2	函数表达式块作为函数体	扩展	✗
7.3	选择表达式	扩展	✗
7.4	循环表达式	扩展	✗
8.1	数组类型	扩展	✓
8.2	数组表达式	扩展	✓
8.3	数组元素访问	扩展	✓
9.1	元组类型	扩展	✓

3 词法分析器设计与实现

词法分析是编译过程的第一阶段，负责将源代码的字符流转换为词法单元序列。本章详细介绍词法分析器的设计与实现。

3.1 词法分析理论基础

词法分析的核心理论基础是有限状态自动机（Finite Automaton, FA）。有限状态自动机是一种数学模型，能够识别正则语言。

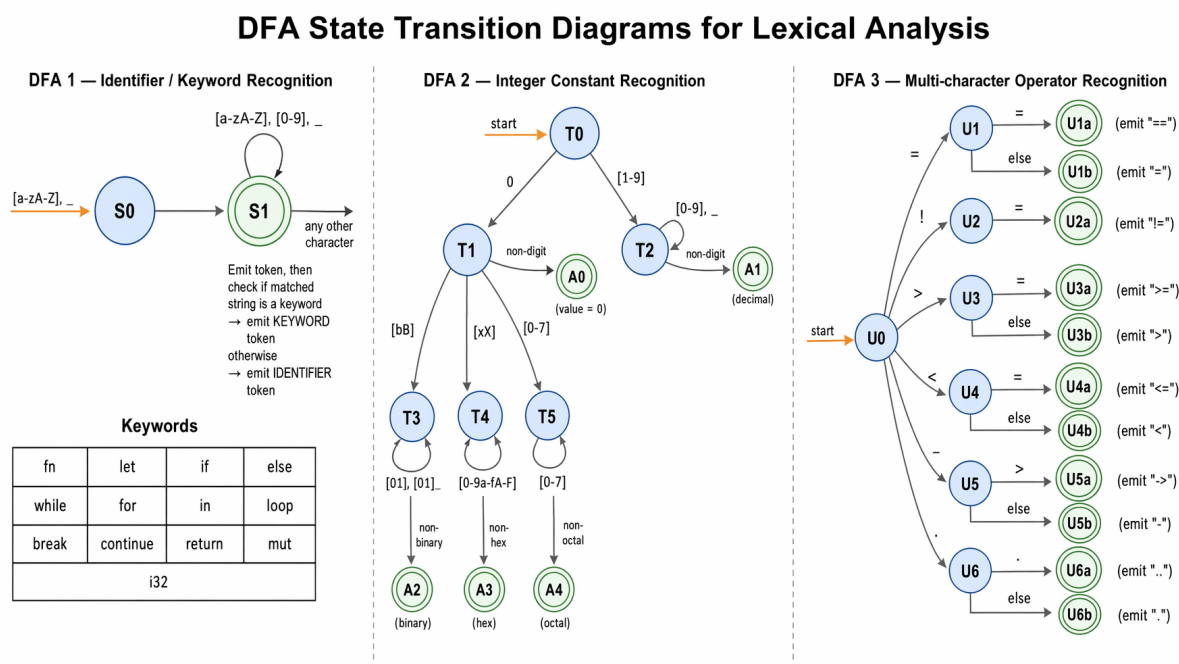


图 2: 词法分析有限状态自动机示意图

Flex 工具接受正则表达式作为输入，自动生成对应的有限状态自动机。正则表达式具有以下性质：

- 能够表示所有正则语言
- 易于书写和理解
- 可通过算法转换为等价的 DFA

3.2 Flex 工具介绍

Flex 是一个快速的词法分析器生成器，是 Unix 系统中 lex 工具的免费替代品。其工作流程如下：

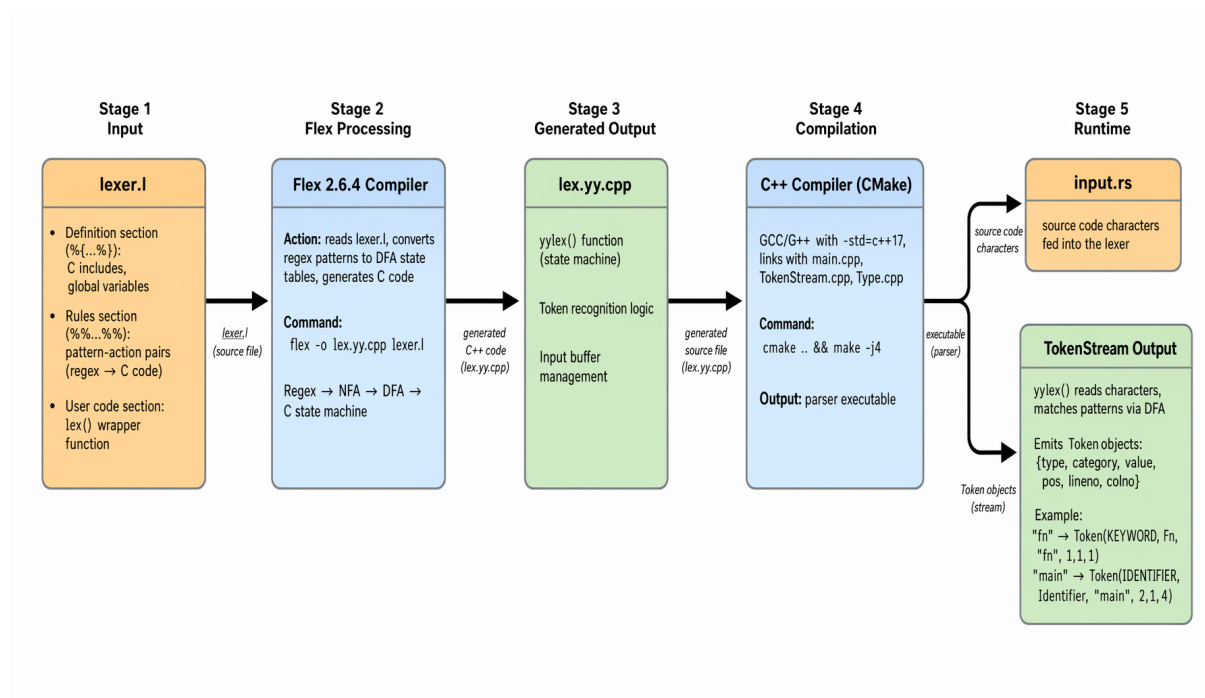


图 3: Flex 工作流程图

1. 编写 Flex 规则文件（.l），定义正则表达式和对应的动作
2. Flex 读取规则文件，生成 C 语言源文件（lex.yy.c）
3. 编译生成的 C 文件，得到词法分析器可执行程序
4. 运行词法分析器，输入源代码，输出 Token 序列

Flex 规则文件的基本结构：

```

1 %{
2 /* C代码声明区：头文件、全局变量、辅助函数 */
3 #include "TokenStream.h"
4 #include "Token.h"
5 %}
6
7 /* Flex选项定义 */
8 %option yylineno
  
```



```
9 %option noyywrap
10
11 /* 正则表达式宏定义 */
12 DIGIT      [0-9]
13 LETTER     [a-zA-Z]
14 ...
15
16 %%
17 /* 规则区：正则表达式 + 动作 */
18 {LETTER}({LETTER}|{DIGIT})*  { /* 标识符动作 */ }
19 ...
20
21 %%
22 /* 用户代码区：辅助函数实现 */
```

Listing 1: Flex 规则文件结构

3.3 Token 设计

Token 是词法分析的基本输出单位，包含类型、种别、值、位置等信息。

3.3.1 Type 枚举定义

```
1 enum class Type {
2     KEYWORD,      // 关键字
3     OPERATOR,     // 运算符
4     DELIMITER,    // 界符
5     CONSTANT,     // 常量
6     IDENTIFIER,   // 标识符
7     DECLERATOR,   // 类型声明符
8     COMMENT,      // 注释
9     TYPE_EOF      // 文件结束
10};
```

Listing 2: Type 枚举定义 (Type.h)

3.3.2 Token 类定义

```
1 class Token {
```

```

2 public:
3     Type type;           // Token 类型
4     string category;     // 种别码
5     string value;        // 原始文本值
6     int pos;             // 在 Token 流中的位置
7     int lineno;          // 行号（用于错误报告）
8     int colno;           // 列号（用于错误报告）
9
10    Token() = default;
11    Token(Type t, const string& cat, const string& val,
12           int p = 0, int ln = 0, int cn = 0)
13        : type(t), category(cat), value(val),
14          pos(p), lineno(ln), colno(cn) {}
15 };

```

Listing 3: Token 类定义 (Token.h)

Token 设计考虑了以下因素：

- 类型区分：通过 Type 枚举快速判断 Token 大类
- 种别码细化：通过 category 区分同类型的不同 Token
- 位置信息：记录行列号，便于语法分析时报错定位

3.4 正则表达式规则设计

3.4.1 常量识别规则

常量识别是词法分析中最复杂的部分，需要支持多种进制和格式。

```

1  /* 数字基础定义 */
2  DIGIT                [0-9]
3  NoneZeroDigit        [1-9]
4  OctalDigit           [0-7]
5  HexadecimalDigit     [0-9a-fA-F]
6  BinaryDigit          [01]
7
8  /* 整数后缀 */
9  IntegerSuffix        (i8|i16|i32|i64|i128|u8|u16|u32|u64|u128|isize|usize)
10
11 /* 各种进制整数 */
12 BinaryConstant        0[bB]({BinaryDigit}+|{BinaryDigit}({BinaryDigit}_)*{BinaryDigit})
13 OctalConstant         0{OctalDigit}+
14 HexadecimalConstant   0[xX]({HexadecimalDigit}+|{HexadecimalDigit}({HexadecimalDigit}_)*{HexadecimalDigit})

```

```

15 DecimalConstant      ({NoneZeroDigit}{DIGIT}*|{NoneZeroDigit}({DIGIT}|_)*{DIGIT}|0)
16
17 /* 整数常量 (合并) */
18 IntegerConstant
19     ({DecimalConstant}{IntegerSuffix}?|{HexadecimalConstant}{IntegerSuffix}?|{BinaryConstant})
20
21 /* 浮点常量 */
22 FractionalConstant    ({DigitSequence}?\. {DigitSequence}|{DigitSequence}\.)
23 ExponentPart          [eE][+-]?{DigitSequence}
24 FloatingSuffix        [fFfL]
25 FloatingConstant      {FractionalConstant}{ExponentPart}?{FloatingSuffix}?
26
27 /* 字符串常量 */
28 SChar                 ([^"\\r\n]|{EscapeSequence}|\\n|\\r)
29 StringConst           \"{SChar}*\"
30
31 /* 字符常量 */
32 CChar                 [^'\\r\n]|{EscapeSequence}
33 CharacterConstant      \"{CChar}+\"

```

Listing 4: 常量正则表达式规则

3.4.2 关键字识别规则

关键字需要优先于标识符识别，以避免误判。

```

1 /* 关键字必须放在标识符规则之前 */
2 let      { CURRENT_TOKEN = Token(Type::KEYWORD, "Let", yytext, ...);
3           return 1; }
4 mut      { CURRENT_TOKEN = Token(Type::KEYWORD, "Mut", yytext, ...);
5           return 1; }
6 fn        { CURRENT_TOKEN = Token(Type::KEYWORD, "Fn", yytext, ...);
7           return 1; }
8 if        { CURRENT_TOKEN = Token(Type::KEYWORD, "If", yytext, ...);
9           return 1; }
10 else     { CURRENT_TOKEN = Token(Type::KEYWORD, "Else", yytext,
11                                   ...); return 1; }
12 while    { CURRENT_TOKEN = Token(Type::KEYWORD, "While", yytext,
13                                   ...); return 1; }
14 for      { CURRENT_TOKEN = Token(Type::KEYWORD, "For", yytext, ...);
15           return 1; }
16 in       { CURRENT_TOKEN = Token(Type::KEYWORD, "In", yytext, ...);
17           return 1; }
18 loop     { CURRENT_TOKEN = Token(Type::KEYWORD, "Loop", yytext,
19                                   ...); return 1; }

```

```

11 break      { CURRENT_TOKEN = Token(Type::KEYWORD, "Break", yytext,
    ...); return 1; }
12 continue { CURRENT_TOKEN = Token(Type::KEYWORD, "Continue", yytext,
    ...); return 1; }
13 return     { CURRENT_TOKEN = Token(Type::KEYWORD, "Return", yytext,
    ...); return 1; }
14
15 /* 类型声明符 */
16 i32        { CURRENT_TOKEN = Token(Type::DECLERATOR, "I32", yytext,
    ...); return 1; }

```

Listing 5: 关键字识别规则

3.4.3 关键字与标识符的区分机制

作业明确要求验证以下边界情况：

```

1 if123      // 应识别为标识符
2 if=123     // 应识别为三个 Token: (if, 关键字) (=, 算符) (123, 整数)

```

Flex 采用**最长匹配**（Maximal Munch）原则：在所有能匹配当前输入的正则表达式中，选择匹配**最长字符串**的那条规则；若长度相同，则选择在规则文件中**先出现**的那条。

对于 if123：

- 关键字规则 if 匹配 2 个字符
- 标识符规则 {*Identifier*} 匹配 5 个字符（if123 整体符合“字母开头、后跟字母数字下划线”的模式）
- 最长匹配原则选择标识符规则，if123 被正确识别为标识符

对于 if=123：

- 首次匹配时，关键字 if 和标识符前缀 i 均可匹配，但 if 之后的 = 不是字母/数字/下划线，因此标识符规则最多匹配 2 个字符（if），与关键字规则长度相同
- 长度相同时，Flex 选择先出现的规则——关键字 if 的规则位于标识符规则之前，因此 if 被识别为关键字
- 随后 = 匹配赋值号界符，123 匹配整数常量

- 最终产生三个 Token:(keyword, If, if)、(delimiter, Assign, =)、(constant, IntegerConstant, 123)

这种机制保证了关键字不会被误识别为标识符前缀,同时也允许以关键字为前缀的合法标识符(如 if123、let_x)被正确识别。

3.4.4 运算符与界符识别

```

1 /* 运算符 (注意顺序: 长运算符优先) */
2 "->"      { CURRENT_TOKEN = Token(Type::OPERATOR, "Arrow", yytext,
3             ...); return 1; }
4 ".."      { CURRENT_TOKEN = Token(Type::OPERATOR, "DotDot", yytext,
5             ...); return 1; }
6 "."       { CURRENT_TOKEN = Token(Type::OPERATOR, "Dot", yytext,
7             ...); return 1; }
8 "=="      { CURRENT_TOKEN = Token(Type::OPERATOR, "EqualEqual",
9             yytext, ...); return 1; }
10 "!="      { CURRENT_TOKEN = Token(Type::OPERATOR, "NotEqual", yytext,
11             ...); return 1; }
12 ">="      { CURRENT_TOKEN = Token(Type::OPERATOR, "GreaterEqual",
13             yytext, ...); return 1; }
14 "<="      { CURRENT_TOKEN = Token(Type::OPERATOR, "LessEqual",
15             yytext, ...); return 1; }
16 ">"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Greater", yytext,
17             ...); return 1; }
18 "<"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Less", yytext,
19             ...); return 1; }
20 "+"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Plus", yytext,
21             ...); return 1; }
22 "-"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Minus", yytext,
23             ...); return 1; }
24 "*"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Star", yytext,
25             ...); return 1; }
26 "/"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Slash", yytext,
27             ...); return 1; }
28 "&"       { CURRENT_TOKEN = Token(Type::OPERATOR, "Ampersand",
29             yytext, ...); return 1; }
30
31 /* 界符 */

```

```

18 "="      { CURRENT_TOKEN = Token(Type::DELIMITER, "Assign", yytext,
...); return 1; }
19 ";"      { CURRENT_TOKEN = Token(Type::DELIMITER, "Semicolon",
yytext, ...); return 1; }
20 ":"      { CURRENT_TOKEN = Token(Type::DELIMITER, "Colon", yytext,
...); return 1; }
21 ","      { CURRENT_TOKEN = Token(Type::DELIMITER, "Comma", yytext,
...); return 1; }
22 "("      { CURRENT_TOKEN = Token(Type::DELIMITER, "LParen", yytext,
...); return 1; }
23 ")"      { CURRENT_TOKEN = Token(Type::DELIMITER, "RParen", yytext,
...); return 1; }
24 "{"      { CURRENT_TOKEN = Token(Type::DELIMITER, "LBrace", yytext,
...); return 1; }
25 "}"      { CURRENT_TOKEN = Token(Type::DELIMITER, "RBrace", yytext,
...); return 1; }
26 "["      { CURRENT_TOKEN = Token(Type::DELIMITER, "LBracket",
yytext, ...); return 1; }
27 "]"      { CURRENT_TOKEN = Token(Type::DELIMITER, "RBracket",
yytext, ...); return 1; }

```

Listing 6: 运算符与界符识别规则

3.5 输出格式设计

词法分析器输出采用 TSV (Tab-Separated Values) 格式, 便于后续处理和可视化。

表 10: 词法分析输出格式示例

Type	Category	Value
keyword	Let	let
keyword	Mut	mut
identifier	Identifier	x
delimiter	Assign	=
constant	IntegerConstant	42
delimiter	Semicolon	;

输出格式设计说明:

- 首行表头: 标注各列含义

- 分隔符：使用制表符（\t），便于程序解析
- 注释过滤：语法分析时自动跳过注释 Token
- 特殊字符转义：对于字符串中的换行等特殊字符，采用自定义转义规则

3.6 TokenStream 接口

TokenStream 类作为词法分析器与语法分析器之间的桥梁，提供流式 Token 访问接口。

```
1 class TokenStream {
2 protected:
3     vector<Token> toks;    // Token 存储
4     int cur = 0;           // 当前位置指针
5
6 public:
7     void load(istream& in);           // 加载 Token 流
8     const Token& peek(int offset = 0); // 查看当前 Token（不移动指针）
9     Token advance();                  // 获取当前 Token 并移动指针
10    Token expect(const string& cat);   // 期望特定种别，否则报错
11    bool check(const string& cat);     // 检查当前 Token 是否匹配
12    bool atEnd();                      // 判断是否到达末尾
13};
```

Listing 7: TokenStream 核心接口

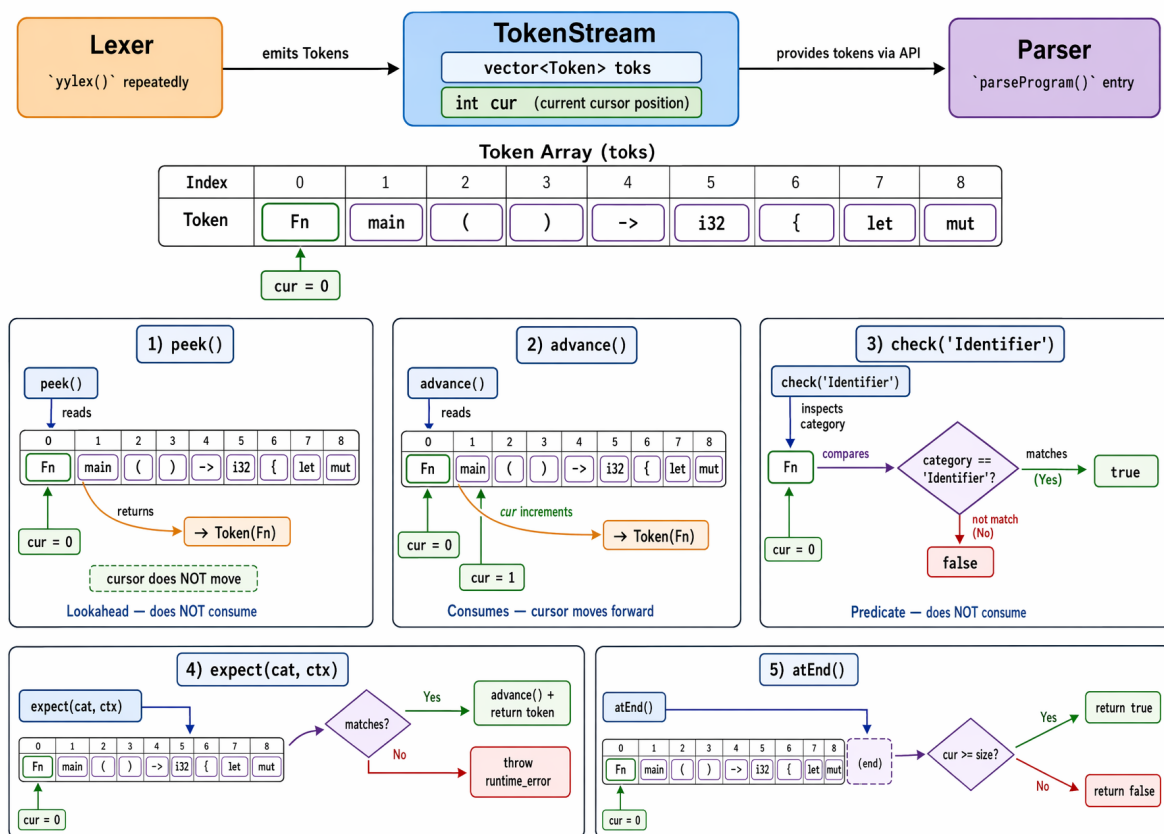


图 4: TokenStream 工作原理图

4 语法分析器设计与实现

语法分析是编译过程的第二阶段，负责将 Token 序列组织成语法树。本章详细介绍递归下降语法分析器的设计与实现。

4.1 语法分析理论基础

4.1.1 递归下降分析法原理

递归下降分析法（Recursive Descent Parsing）是一种自顶向下的语法分析方法，其核心思想是：

- 为每个非终结符编写一个对应的递归函数
- 函数根据当前 Token 选择匹配的的产生式
- 通过递归调用处理嵌套结构

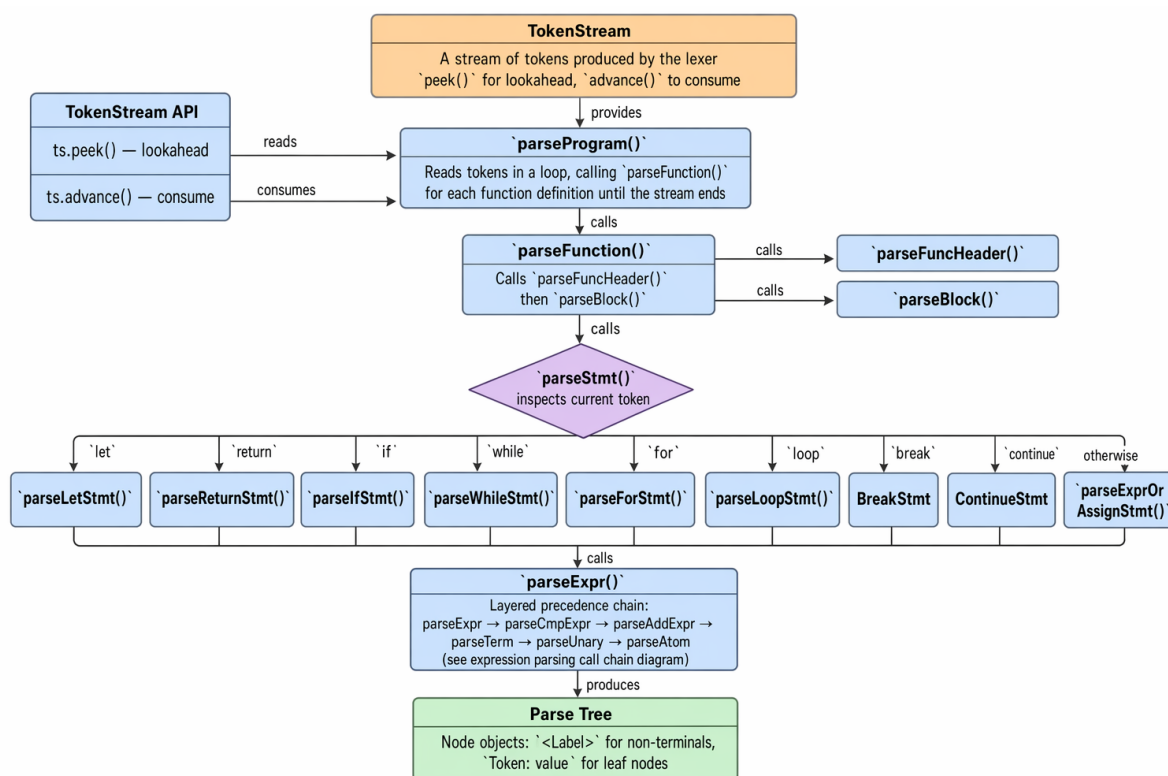


图 5: 递归下降分析流程图

递归下降分析的特点：

1. 优点:

- 实现简单直观, 易于理解和调试
- 可以灵活添加错误处理和恢复机制
- 便于扩展和修改文法规则

2. 局限性:

- 要求文法满足 LL(1) 条件
- 需要消除左递归
- 对某些复杂文法处理困难

4.1.2 LL(1) 文法判定

LL(1) 文法是指可以从左向右扫描输入、进行最左推导、仅使用 1 个向前查看符号的文法。判断文法是否为 LL(1) 需满足:

1. 无左递归
2. 无公共左因子
3. 对于每个非终结符 A 的两个产生式 $A \rightarrow \alpha|\beta$, 满足 $FIRST(\alpha) \cap FIRST(\beta) = \emptyset$

4.1.3 左递归消除方法

左递归是指形如 $A \rightarrow A\alpha|\beta$ 的产生式。直接左递归可通过以下公式消除:

设 $A \rightarrow A\alpha_1|A\alpha_2|\dots|A\alpha_n|\beta_1|\beta_2|\dots|\beta_m$

转换为:

$$A \rightarrow \beta_1 A'|\beta_2 A'|\dots|\beta_m A' \quad (99)$$

$$A' \rightarrow \alpha_1 A'|\alpha_2 A'|\dots|\alpha_n A'|\epsilon \quad (100)$$

例如, 表达式文法 $E \rightarrow E + T|T$ 消除左递归后:

$$E \rightarrow TE' \quad (101)$$

$$E' \rightarrow +TE'|\epsilon \quad (102)$$

4.2 语法树数据结构

语法树 (Parse Tree) 是语法分析的输出结果, 反映了源代码的语法结构。

4.2.1 Node 类定义

```
1 class Node {
2 public:
3     string label;                // 节点标签 (如 "IfStmt",
        "Identifier")
4     vector<unique_ptr<Node>> children; // 子节点列表
5     bool isLeaf = false;        // 是否为叶节点
6
7     explicit Node(string lbl, bool leaf = false)
8         : label(move(lbl)), isLeaf(leaf) {}
9
10    void PrintToFile(ofstream& ofs, int depth = 0) const;
11};
```

Listing 8: Node 类定义

4.2.2 辅助函数

```
1 using NodePtr = unique_ptr<Node>;
2
3 // 创建非叶节点
4 static inline NodePtr makeNode(const string& lbl) {
5     return make_unique<Node>(lbl);
6 }
7
8 // 创建叶节点 (Token)
9 static inline NodePtr makeLeaf(const string& lbl) {
10    return make_unique<Node>(lbl, true);
11 }
12
13 // 添加子节点
14 static inline void addChild(NodePtr& parent, NodePtr child) {
15    parent->children.push_back(move(child));
16 }
```

Listing 9: 语法树辅助函数

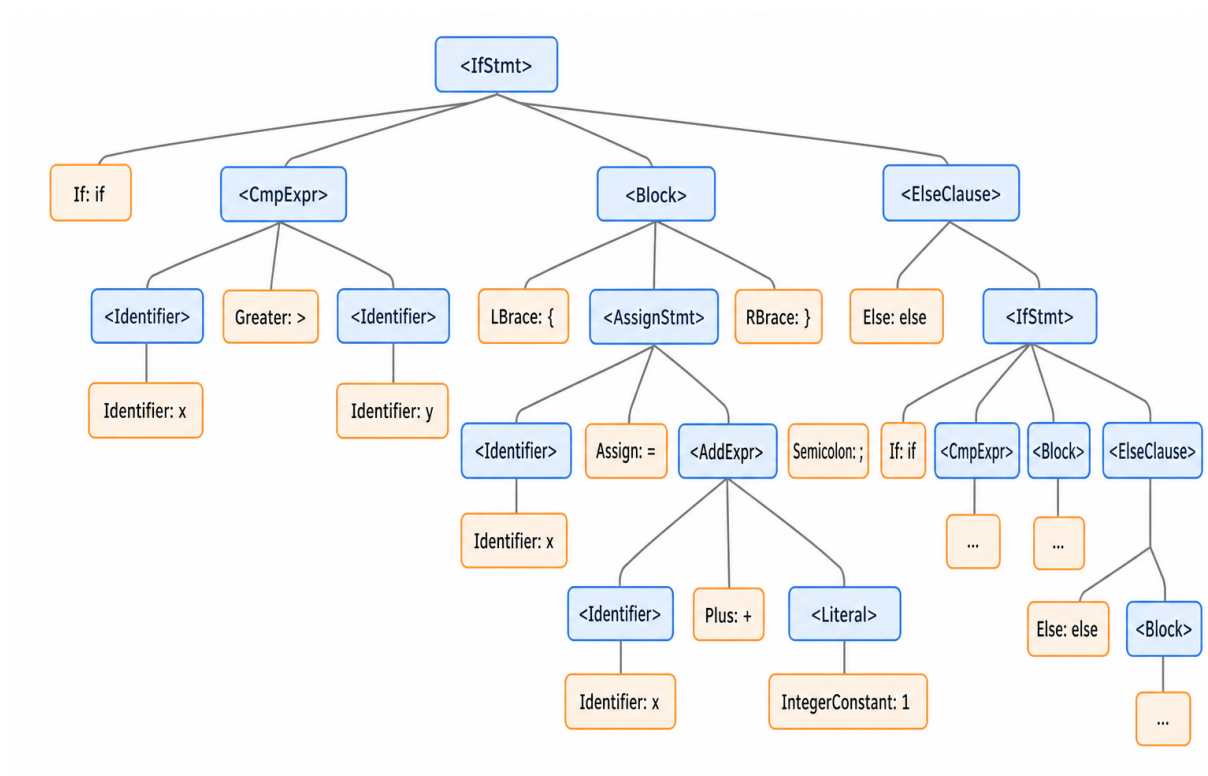


图 6: 语法树结构示意图 (if-else 语句)

4.3 Parser 类实现

Parser 类是语法分析器的核心，封装了所有解析函数。

```

1 class Parser {
2     TokenStream& ts; // Token 流引用
3
4     // 辅助函数
5     NodePtr consumeLeaf(); //
6     // 消耗 Token, 生成叶节点
7     NodePtr expectLeaf(const string& cat); // 期望特定 Token
8 public:
9     explicit Parser(TokenStream& ts) : ts(ts) {}
10
11     NodePtr parse(); // 入口: 解析整个程序
12     NodePtr parseProgram(); // 解析程序
13     NodePtr parseFunction(); // 解析函数
14     NodePtr parseStmt(); // 解析语句 (分发)

```

```

15     NodePtr parseExpr();           // 解析表达式
16     // ... 其他解析函数
17 };

```

Listing 10: Parser 类框架

4.3.1 核心解析函数示例

if 语句解析函数:

```

1 NodePtr parseIfStmt() {
2     auto node = makeNode("IfStmt");
3     addChild(node, expectLeaf("If"));           // if关键字
4     addChild(node, parseExpr());               // 条件表达式
5     addChild(node, parseBlock());              // if块
6
7     if (ts.check("Else")) {
8         auto elseNode = makeNode("ElseClause");
9         addChild(elseNode, consumeLeaf());      // else关键字
10        if (ts.check("If")) {
11            addChild(elseNode, parseIfStmt());  // else-if
12        } else {
13            addChild(elseNode, parseBlock());   // else块
14        }
15        addChild(node, move(elseNode));
16    }
17    return node;
18 }

```

Listing 11: parseIfStmt 函数实现

let 语句解析函数:

```

1 NodePtr parseLetStmt() {
2     auto node = makeNode("LetStmt");
3     addChild(node, expectLeaf("Let"));
4
5     // 变量声明
6     auto decl = makeNode("VarDecl");
7     if (ts.check("Mut")) addChild(decl, consumeLeaf()); // [mut]
8     addChild(decl, expectLeaf("Identifier", "variable name"));
9     if (ts.check("Colon")) {

```

```

10     addChild(decl, consumeLeaf());
11     addChild(decl, parseType()); // [: Type]
12 }
13 addChild(node, move(decl));
14
15 // 初始化表达式
16 if (ts.check("Assign")) {
17     addChild(node, consumeLeaf());
18     addChild(node, parseExpr()); // [= Expr]
19 }
20 addChild(node, expectLeaf("Semicolon"));
21 return node;
22 }

```

Listing 12: parseLetStmt 函数实现

语句分发函数:

```

1 NodePtr parseStmt() {
2     const Token& tok = ts.peek();
3
4     if (tok.category == "Semicolon") return parseEmptyStmt();
5     if (tok.category == "Let") return parseLetStmt();
6     if (tok.category == "Return") return parseReturnStmt();
7     if (tok.category == "If") return parseIfStmt();
8     if (tok.category == "While") return parseWhileStmt();
9     if (tok.category == "For") return parseForStmt();
10    if (tok.category == "Loop") return parseLoopStmt();
11    if (tok.category == "Break") return parseBreakStmt();
12    if (tok.category == "Continue") return parseContinueStmt();
13    if (tok.category == "LBrace") return parseBlock();
14
15    // 其他: 赋值语句或表达式语句
16    return parseExprOrAssignStmt();
17 }

```

Listing 13: parseStmt 语句分发

4.4 表达式解析与运算符优先级

表达式解析需要正确处理运算符优先级。本项目采用分层递归的方法，每层对应一个优先级。

表 11: 运算符优先级层次

层次	函数名	运算符	优先级
1	parseExpr	.. (范围)	最低
2	parseCmpExpr	== != > >= < <=	低
3	parseAddExpr	+ -	中低
4	parseTerm	* /	中高
5	parseUnary	& * &mut (单目)	高
6	parseAtom	常量、标识符等	最高

4.4.1 表达式解析函数

```

1 // Expr -> CmpExpr ['..' CmpExpr]
2 NodePtr parseExpr() {
3     NodePtr lhs = parseCmpExpr();
4     if (ts.check("DotDot")) {
5         auto node = makeNode("RangeExpr");
6         addChild(node, move(lhs));
7         addChild(node, consumeLeaf());
8         addChild(node, parseCmpExpr());
9         return node;
10    }
11    return lhs;
12 }

13
14 // CmpExpr -> AddExpr { CmpOp AddExpr }
15 NodePtr parseCmpExpr() {
16     NodePtr lhs = parseAddExpr();
17     while (isCmpOp()) { // == != > >= < <=
18         auto node = makeNode("CmpExpr");
19         addChild(node, move(lhs));
20         addChild(node, consumeLeaf());
21         addChild(node, parseAddExpr());
22         lhs = move(node);
23     }

```

```

24     return lhs;
25 }
26
27 // AddExpr -> Term { ('+'|'-') Term }
28 NodePtr parseAddExpr() {
29     NodePtr lhs = parseTerm();
30     while (ts.check("Plus") || ts.check("Minus")) {
31         auto node = makeNode("AddExpr");
32         addChild(node, move(lhs));
33         addChild(node, consumeLeaf());
34         addChild(node, parseTerm());
35         lhs = move(node);
36     }
37     return lhs;
38 }
39
40 // Term -> Unary { ('*'|'/') Unary }
41 NodePtr parseTerm() {
42     NodePtr lhs = parseUnary();
43     while (ts.check("Star") || ts.check("Slash")) {
44         auto node = makeNode("MulExpr");
45         addChild(node, move(lhs));
46         addChild(node, consumeLeaf());
47         addChild(node, parseUnary());
48         lhs = move(node);
49     }
50     return lhs;
51 }
52
53 // Unary -> '&' ['mut'] Unary / '*' Unary / Atom
54 NodePtr parseUnary() {
55     if (ts.check("Ampersand")) {
56         auto node = makeNode("RefExpr");
57         addChild(node, consumeLeaf());
58         if (ts.check("Mut")) addChild(node, consumeLeaf());
59         addChild(node, parseUnary());
60         return node;
61     }
62     if (ts.check("Star")) {
63         auto node = makeNode("DerefExpr");

```



```

64     addChild(node, consumeLeaf());
65     addChild(node, parseUnary());
66     return node;
67 }
68 return parseAtom();
69 }

```

Listing 14: 表达式解析函数系列

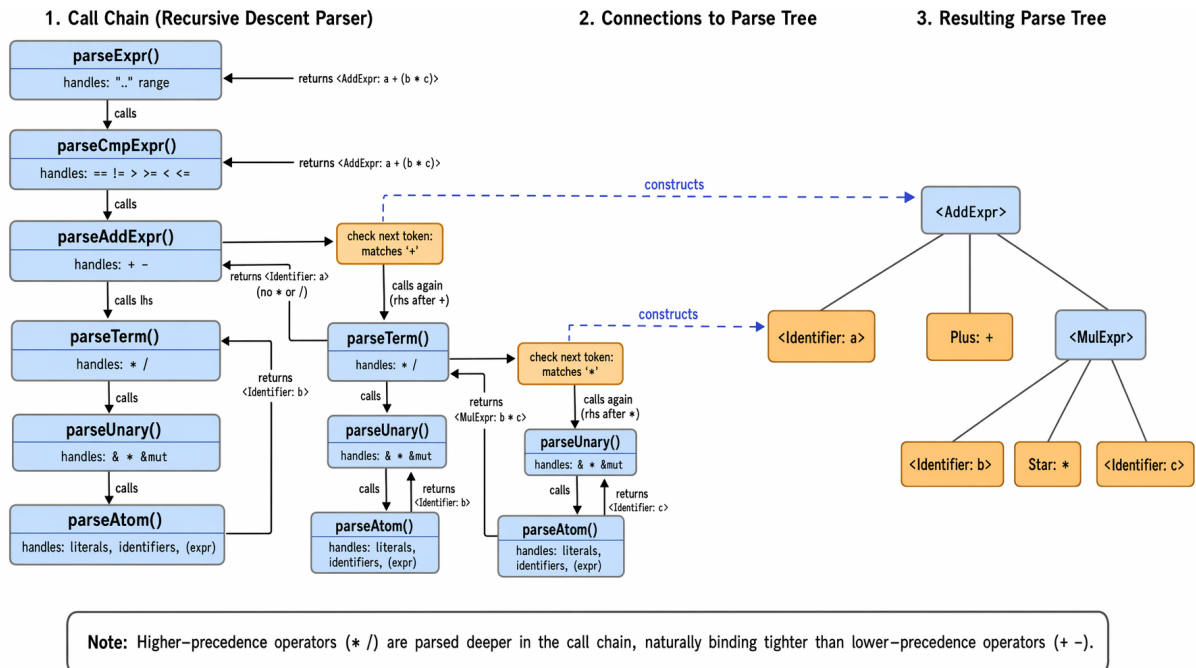


图 7: 表达式解析调用链图

5 GUI 可视化模块

为了提供直观的分析结果展示，本项目基于 ImGui 框架实现了图形化界面。本章介绍 GUI 模块的设计与实现。

5.1 ImGui 框架简介

ImGui (Immediate Mode GUI) 是一个轻量级的 C++ GUI 库，具有以下特点：

- **即时模式：** 界面状态在每帧重新计算，无需持久化存储
- **易于集成：** 可直接嵌入现有渲染管线
- **跨平台：** 支持多种后端 (OpenGL、Vulkan、DirectX 等)
- **调试友好：** 常用于游戏开发调试工具

本项目使用：

- ImGui 核心库
- GLFW 窗口管理
- OpenGL3 渲染后端

5.2 界面布局设计

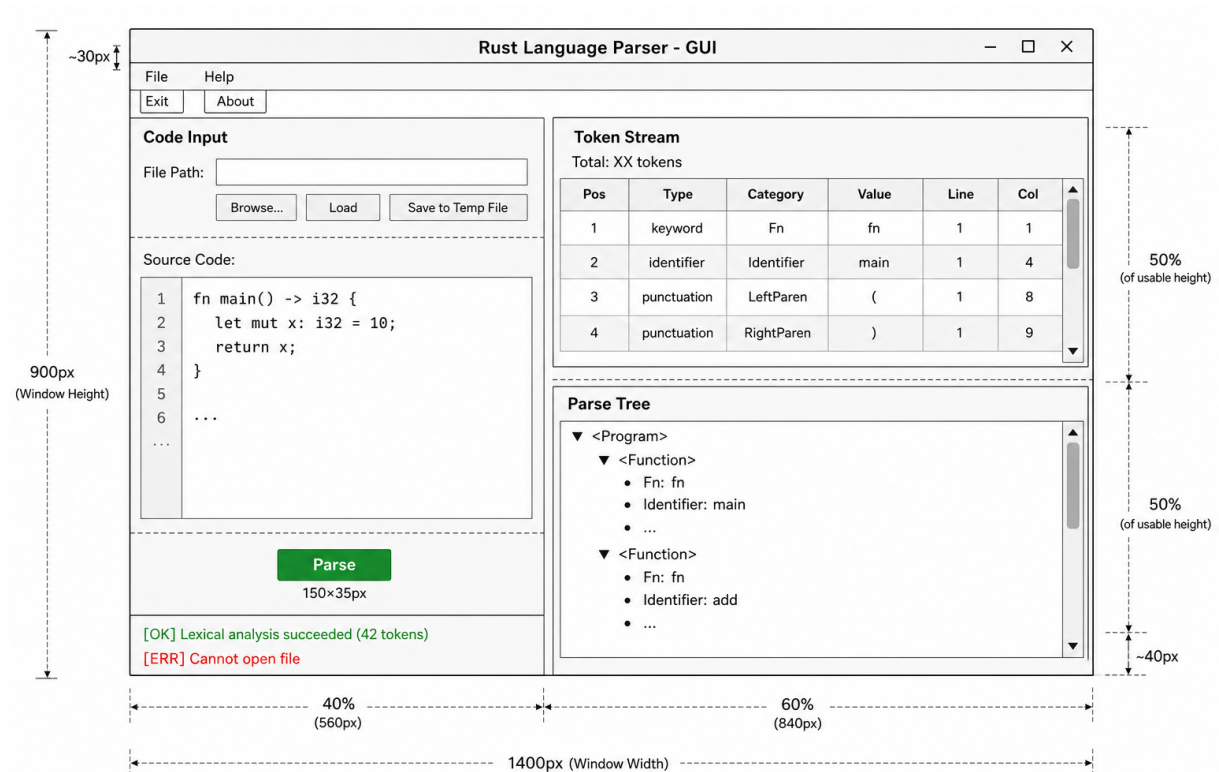


图 8: GUI 界面布局设计图

5.3 核心组件实现

5.3.1 Token 列表展示

```
1 static void render_token_list() {
2     ImGui::Begin("Token Stream");
3     ImGui::Text("Total: %zu tokens", g_state.tokens.size());
4
5     if (ImGui::BeginTable("Tokens", 6,
6         ImGuiTableFlags_Borders | ImGuiTableFlags_RowBg)) {
7         ImGui::TableSetupColumn("Pos");
8         ImGui::TableSetupColumn("Type");
9         ImGui::TableSetupColumn("Category");
10        ImGui::TableSetupColumn("Value");
11        ImGui::TableSetupColumn("Line");
12        ImGui::TableSetupColumn("Col");
13        ImGui::TableHeadersRow();
```

```
14
15     for (const auto& tok : g_state.tokens) {
16         ImGui::TableNextRow();
17         ImGui::TableSetColumnIndex(0);
18         ImGui::Text("%d", tok.pos);
19         // ... 其他列
20     }
21     ImGui::EndTable();
22 }
23 ImGui::End();
24 }
```

Listing 15: Token 列表渲染

5.3.2 语法树渲染

```
1 static void render_tree_node(const Node& node, const void* node_ptr)
2 {
3     if (node.isLeaf) {
4         ImGuiTreeNodeFlags flags = ImGuiTreeNodeFlags_Leaf
5             | ImGuiTreeNodeFlags_NoTreePushOnOpen
6             | ImGuiTreeNodeFlags_Bullet;
7         ImGui::TreeNodeEx(node.label.c_str(), flags);
8     } else {
9         ImGuiTreeNodeFlags flags = ImGuiTreeNodeFlags_OpenOnArrow
10             | ImGuiTreeNodeFlags_OpenOnDoubleClick;
11         bool node_open = ImGui::TreeNodeEx(node.label.c_str(),
12             flags);
13         if (node_open) {
14             for (const auto& child : node.children) {
15                 render_tree_node(*child, child.get());
16             }
17             ImGui::TreePop();
18         }
19     }
20 }
```

Listing 16: 语法树递归渲染

5.4 GUI 架构

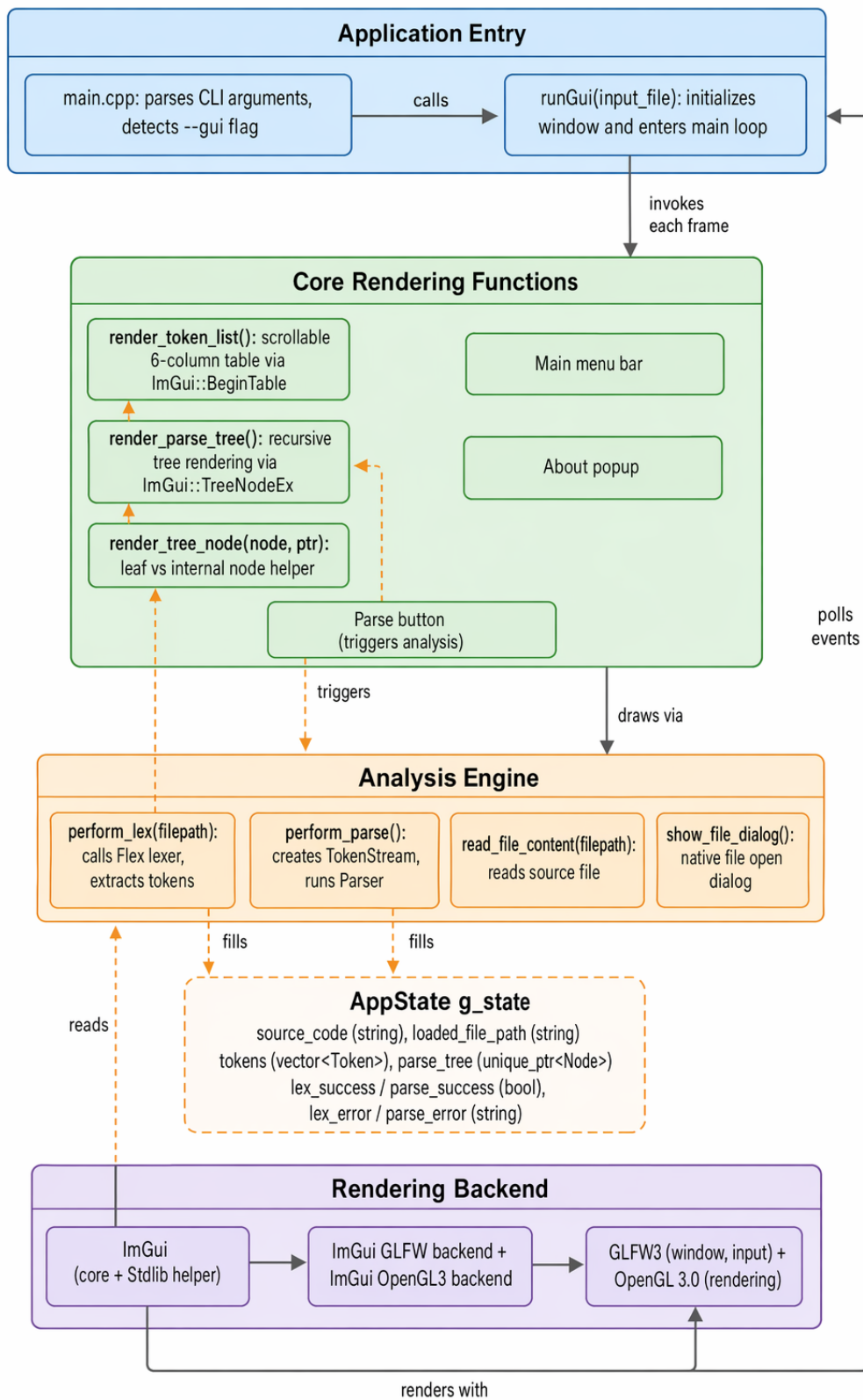


图 9: GUI 模块架构图

6 构建与跨平台支持

本项目采用 CMake 构建系统，支持 Linux 本地编译和 Windows 交叉编译。

6.1 CMake 构建系统

CMake 是一个跨平台的构建工具，能够生成各种编译系统的配置文件。

```
1 cmake_minimum_required(VERSION 3.10)
2 project(Parser VERSION 1.0 LANGUAGES CXX)
3
4 # C++17 标准
5 set(CMAKE_CXX_STANDARD 17)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7
8 # 源文件
9 set(PARSER_SOURCES
10     src/main.cpp
11     src/TokenStream.cpp
12     src/Type.cpp
13     src/lex.yy.cpp # Flex 生成
14 )
15
16 # 构建目标
17 add_executable(parser ${PARSER_SOURCES})
```

Listing 17: CMakeLists.txt 核心配置

6.2 Linux/Windows 交叉编译配置

项目支持在 Linux 环境下编译 Windows 可执行文件，使用 MinGW-w64 工具链。

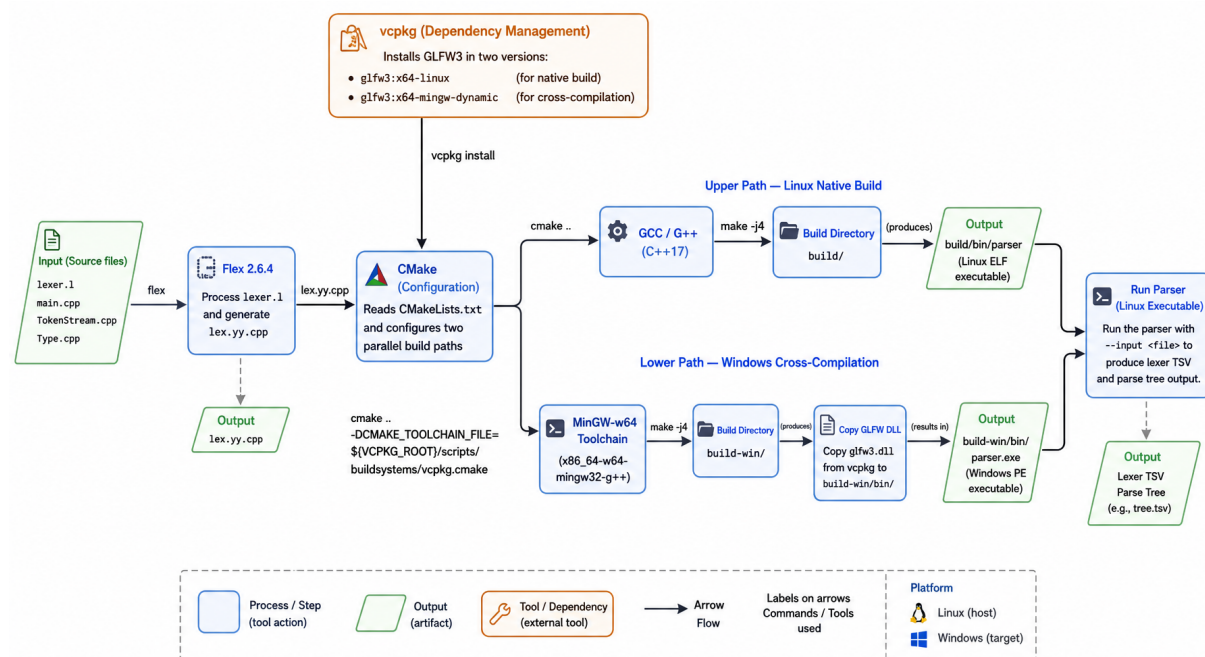


图 10: 跨平台构建流程图

6.3 运行脚本说明

项目提供 `run.sh` 脚本，实现一键构建运行：

```

1 #!/bin/bash
2
3 # 1. 环境检查与依赖安装
4 check_dependencies() {
5     # 检查 flex、g++、cmake、mingw-w64、vcpkg
6 }
7
8 # 2. Flex 代码生成
9 flex -o src/lex.yy.cpp src/lexer.l
10
11 # 3. Linux 版本编译
12 mkdir -p build && cd build
13 cmake ..
14 make -j4
15
16 # 4. Windows 版本编译
17 mkdir -p build-win && cd build-win

```

```
18 cmake .. -DCMAKE_TOOLCHAIN_FILE=/usr/share/cmake/mingw-w64/...
19 make -j4
20
21 # 5. 执行分析
22 ./build/bin/parser --input ./testfiles/input.rs
```

Listing 18: run.sh 脚本核心流程

7 测试与结果分析

本章通过完整的测试用例验证系统的正确性，展示词法分析和语法分析的输出结果。

7.1 测试用例设计

测试用例涵盖类 Rust 语言的主要语法特性：

```
1 // this is a single line comment
2 /* this is a block comment */
3
4 fn main() -> i32 {
5     let mut x: i32 = 10;
6     let y: i32 = 20;
7     let z: i32 = 30;
8
9     // test if-else
10    if x > y {
11        x = x + 1;
12    } else if x < y {
13        x = x - 1;
14    } else {
15        x = 0;
16    }
17
18    // test while loop
19    while x < 15 {
20        x = x + 1;
21    }
22
23    // test for-in loop
24    for i in 0..10 {
25        if i == 5 {
26            break;
27        }
28        continue;
29    }
30
31    // test loop loop
```

```
32     loop {
33         x = x + 1;
34         if x >= 20 {
35             break;
36         }
37     }
38
39     // test operators
40     let a: i32 = x + y;
41     let b: i32 = x - y;
42     let c: i32 = x * y;
43     let d: i32 = x / y;
44
45     // test comparison operators
46     let e: bool = x == y;
47     let f: bool = x != y;
48     let g: bool = x >= y;
49     let h: bool = x <= y;
50
51     // test arrays
52     let arr: [i32; 3] = [1, 2, 3];
53     let val: i32 = arr[0];
54
55     // test function calls
56     let result: i32 = add(x, y);
57
58     // test return
59     return result;
60 }
61
62 fn add(a: i32, b: i32) -> i32 {
63     let sum: i32 = a + b;
64     return sum;
65 }
66
67 fn test_special() -> i32 {
68     let x: i32 = 100;
69     let y: i32 = x;
70     let z: i32 = y;
71 }
```

```

72 // test . and ..
73 let range = 0..100;
74
75 return x;
76 }
77
78 #

```

Listing 19: 完整测试输入 (input.rs)

7.2 词法分析输出示例

以下是词法分析器对 `main` 函数首部的输出：

表 12: 词法分析输出示例 (main 函数首部)

Type	Category	Value	Pos
keyword	Fn	fn	3
identifier	Identifier	main	4
delimiter	LParen	(	5
delimiter	RParen	)	6
operator	Arrow	->	7
declerator	I32	i32	8
delimiter	LBrace	{	9
keyword	Let	let	10
keyword	Mut	mut	11
identifier	Identifier	x	12
delimiter	Colon	:	13
declerator	I32	i32	14
delimiter	Assign	=	15
constant	IntegerConstant	10	16
delimiter	Semicolon	;	17

7.3 语法树输出示例

以下是语法树的部分输出（缩进格式）：

```

1 <IfStmt>
2   If: if

```

```
3  <CmpExpr>
4    <Identifier>
5      Identifier: x
6    Greater: >
7    <Identifier>
8      Identifier: y
9  <Block>
10    LBrace: {
11    <AssignStmt>
12      <Identifier>
13        Identifier: x
14      Assign: =
15      <AddExpr>
16        <Identifier>
17          Identifier: x
18        Plus: +
19        <Literal>
20          IntegerConstant: 1
21      Semicolon: ;
22    RBrace: }
23  <ElseClause>
24    Else: else
25  <IfStmt>
26    If: if
27    <CmpExpr>
28      <Identifier>
29        Identifier: x
30      Less: <
31      <Identifier>
32        Identifier: y
33    <Block>
34      ...
```

Listing 20: 语法树输出示例 (IfStmt 节点)

7.4 分规则测试用例与结果

为验证每条文法规则的正确性，我们按规则分组编写了独立的测试文件。测试文件位于 `parser/testfiles/` 目录下，命名格式为 `test_{规则号}.rs`。以下展示各组的测

试输入及分析结果。

7.4.1 程序结构测试（规则 1.1 – 1.5）

测试文件 test_1_1.rs 至 test_1_5.rs 分别对应规则 1.1–1.5:

```
1 fn program_1_1() {  
2  
3 }
```

Listing 21: test_1_1.rs – 空函数（规则 1.1）

```
1 fn program_1_2() {  
2     ;;;;  
3 }
```

Listing 22: test_1_2.rs – 空语句（规则 1.2）

```
1 fn program_1_3() {  
2     return ;  
3 }
```

Listing 23: test_1_3.rs – 返回语句（规则 1.3）

```
1 fn program_1_4(mut a:i32) {  
2  
3 }
```

Listing 24: test_1_4.rs – 函数输入：形参列表（规则 1.4）

```
1 fn program_1_5() -> i32 {  
2     return 1;  
3 }
```

Listing 25: test_1_5.rs – 函数输出：返回类型（规则 1.5）

测试结果：五个函数均通过语法分析。

7.4.2 变量声明与赋值测试（规则 2.0 – 2.3）

测试文件 test_2.rs 包含三个函数，分别测试规则 2.0–2.3:

```
1 fn program_2_1() {  
2     let mut a;
```

```
3     let mut b:i32;
4 }
5
6 fn program_2_2(mut a:i32) {
7     a=32;
8 }
9
10 fn program_2_3() {
11     let mut a=1;
12     let mut b:i32=1;
13 }
```

Listing 26: test_2.rs – 变量声明与赋值（规则 2.0–2.3）

测试结果：三个函数均通过语法分析，语法树正确生成了 LetStmt、AssignStmt 节点。

7.4.3 表达式测试（规则 3.1 – 3.5）

测试文件 test_3.rs 覆盖了基本表达式、比较运算、加减运算、乘除运算和函数调用：

```
1 fn program_3_1__1() {
2     0;
3     (1);
4     ((2));
5     (((3)));
6 }
7 fn program_3_1__2(mut a:i32) {
8     a;
9     (a);
10    ((a));
11    (((a)));
12 }
13 fn program_3_2() {
14     1<2;
15     3>4;
16 }
17 fn program_3_3() {
18     1+2;
19     3-4;
```

```
20 }
21 fn program_3_4() {
22     1*2;
23     3/4;
24 }
25
26 fn program_3_5__1() {
27
28 }
29 fn program_3_5__2() {
30     program_3_5__1();
31 }
```

Listing 27: test_3.rs – 表达式（规则 3.1–3.5）

测试结果：全部通过。括号嵌套、比较运算、算术运算和函数调用均被正确解析。

7.4.4 选择结构测试（规则 4.1 – 4.3）

测试文件 test_4.rs 覆盖了 if、if-else 和 else if 链：

```
1 fn program_4_1(a:i32) -> i32 {
2     if a>0 {
3         return 1;
4     }
5 }
6 fn program_4_2(a:i32) -> i32 {
7     if a>0 {
8         return 1;
9     } else {
10        return 0;
11    }
12 }
13
14 fn program_4_3(a:i32) -> i32 {
15     if a>0 {
16         return a+1;
17     } else if a<0 {
18         return a-1;
19     } else {
20         return 0;
```

```
21     }  
22 }
```

Listing 28: test_4.rs – 选择结构（规则 4.1–4.3）

测试结果：三个函数均通过。语法分析器正确处理了 `else if` 的递归嵌套。

7.4.5 循环结构测试（规则 5.0 – 5.4）

测试文件 `test_5.rs` 覆盖了 `while`、`for`、`loop` 和 `break/continue`：

```
1 fn program_5_1(mut n:i32) {  
2     while n>0 {  
3         n=n-1;  
4     }  
5 }  
6 fn program_5_2(mut n:i32) {  
7     for mut i in 1..n+1 {  
8         n=n-1;  
9     }  
10 }  
11  
12 fn program_5_3() {  
13     loop {  
14  
15     }  
16 }  
17 fn program_5_4() {  
18     while 1==0 { continue; }  
19     while 1==1 { break; }  
20 }
```

Listing 29: test_5.rs – 循环结构（规则 5.0–5.4）

测试结果：四个函数均通过。`for` 循环的范围表达式 `1..n+1` 被正确解析为 `RangeExpr` 节点。

7.4.6 引用与借用测试（规则 6.1 – 6.4）

测试文件 `test_6.rs` 覆盖了不可变变量、不可变引用、可变引用和解引用：

```
1 fn program_6_1() {
```



```

2      let a:i32;
3      let b;
4      let c:i32=1;
5      let d=2;
6  }
7
8  fn program_6_2(a:i32) {
9      let b:& i32=&a;
10 }
11
12 fn program_6_3(mut a:i32) {
13     let mut b:&mut i32=&mut a;
14 }
15
16 fn program_6_4(a:&mut i32) {
17     let b=*a;
18     *a=3;
19 }

```

Listing 30: test_6.rs – 引用与借用（规则 6.1–6.4）

测试结果：四个函数均通过。解引用赋值 `*a=3`；被正确解析为 `AssignStmt`，左部为 `DerefExpr`。

7.4.7 数组测试（规则 8.1 – 8.3）

测试文件 `test_8.rs` 覆盖了数组类型、数组表达式和数组元素访问：

```

1  fn program_8_1() {
2      let mut a:[i32;3];           // 规则 8.1: 数组类型
3  }
4  fn program_8_2(mut a:[i32;3]) {
5      a=[1,2,3];                  // 规则 8.2: 数组表达式
6  }
7  fn program_8_3(mut a:[i32;3]) {
8      let mut b:i32=a[0];         // 规则 8.3: 数组元素访问
9      a[0]=1;                     // 规则 8.3 + 2.2: 数组元素赋值
10 }

```

Listing 31: 数组测试输入

测试结果：三个函数均通过。

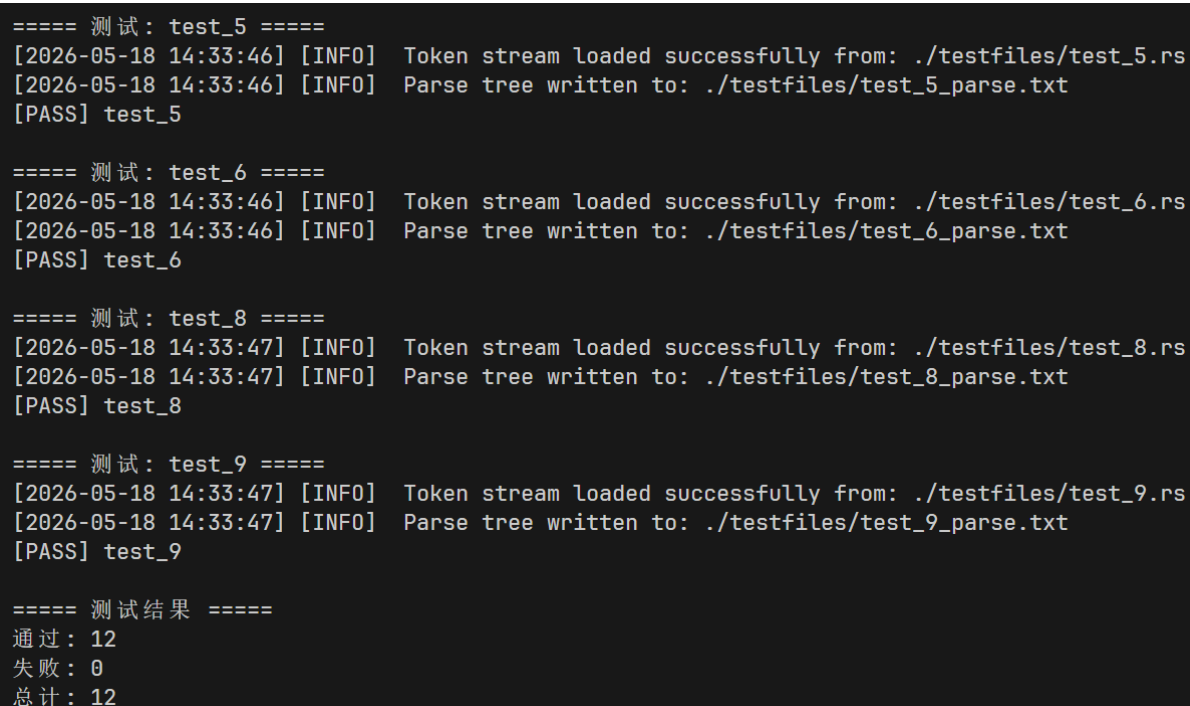
7.4.8 元组类型测试（规则 9.1）

```
1 fn program_9_1() {  
2     let a:(i32,i32);           // 规则 9.1: 元组类型  
3 }
```

Listing 32: 元组类型测试输入

测试结果：通过。元组类型 (i32, i32) 被正确解析。

7.5 运行效果截图



```
==== 测试: test_5 ====  
[2026-05-18 14:33:46] [INFO] Token stream loaded successfully from: ./testfiles/test_5.rs  
[2026-05-18 14:33:46] [INFO] Parse tree written to: ./testfiles/test_5_parse.txt  
[PASS] test_5  
  
==== 测试: test_6 ====  
[2026-05-18 14:33:46] [INFO] Token stream loaded successfully from: ./testfiles/test_6.rs  
[2026-05-18 14:33:46] [INFO] Parse tree written to: ./testfiles/test_6_parse.txt  
[PASS] test_6  
  
==== 测试: test_8 ====  
[2026-05-18 14:33:47] [INFO] Token stream loaded successfully from: ./testfiles/test_8.rs  
[2026-05-18 14:33:47] [INFO] Parse tree written to: ./testfiles/test_8_parse.txt  
[PASS] test_8  
  
==== 测试: test_9 ====  
[2026-05-18 14:33:47] [INFO] Token stream loaded successfully from: ./testfiles/test_9.rs  
[2026-05-18 14:33:47] [INFO] Parse tree written to: ./testfiles/test_9_parse.txt  
[PASS] test_9  
  
==== 测试结果 ====  
通过: 12  
失败: 0  
总计: 12
```

图 11: 命令行批量测试运行截图（12 个测试全部通过）

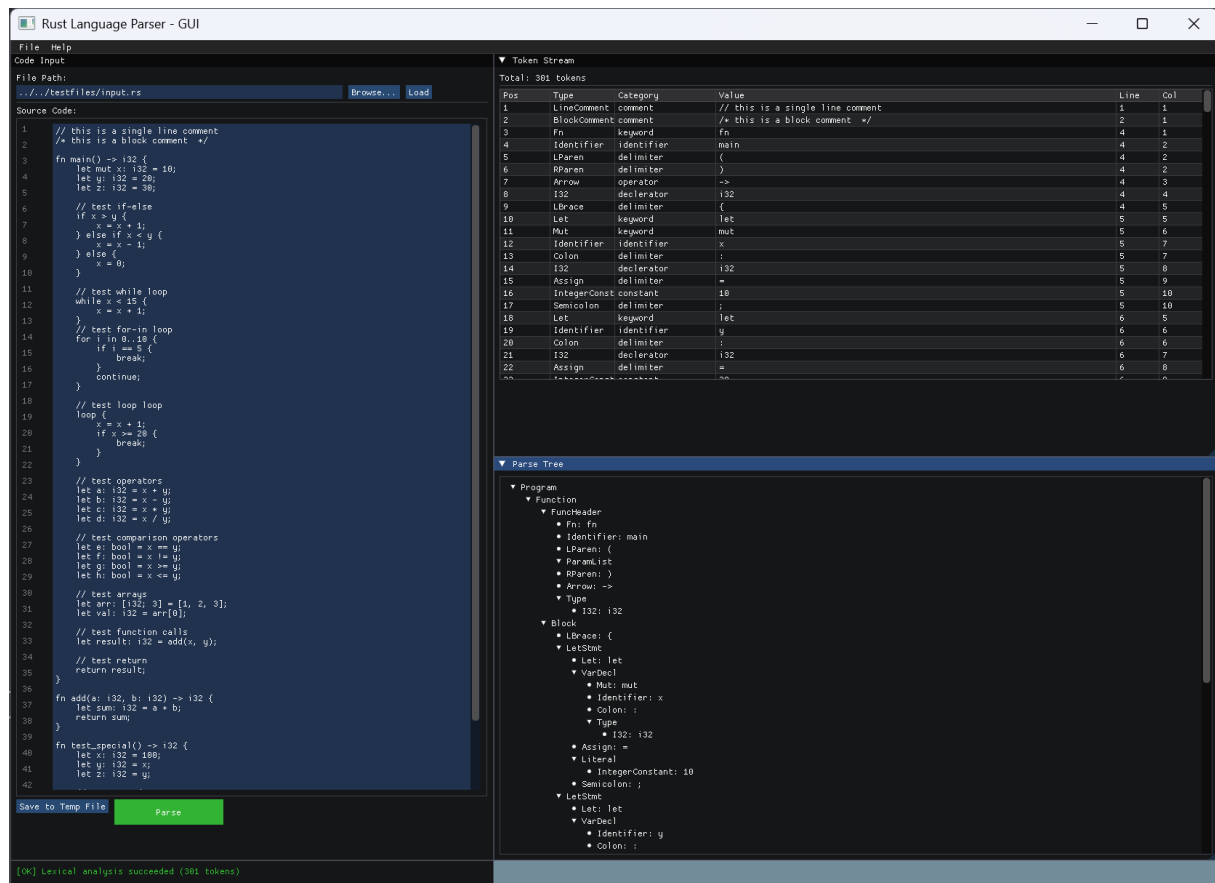


图 12: GUI 界面运行截图：代码输入、Token 表格与语法树可视化

8 思考与分析

作业最后提出了两个思考题，本节分别进行分析和讨论。

8.1 思考题一：1.4 函数输入中的形参列表可以识别怎样的语言？

规则 1.4 定义了函数形参列表的文法：

$$\langle \text{形参列表} \rangle \rightarrow \epsilon \mid \langle \text{形参} \rangle \mid \langle \text{形参} \rangle , \langle \text{形参列表} \rangle \quad (103)$$

$$\langle \text{形参} \rangle \rightarrow \langle \text{变量属性} \rangle \text{ ID} : \langle \text{类型} \rangle \quad (104)$$

其中 $\langle \text{变量属性} \rangle$ 由规则 0.1 和 6.1 定义为 `mut` 或 ϵ ， $\langle \text{类型} \rangle$ 由规则 0.2 定义为 `i32`（后续扩展规则增加数组类型、引用类型、元组类型等）。

语言描述。该文法生成的语言为：零个或多个形参组成的逗号分隔列表，每个形参由可选的 `mut` 修饰符、标识符、冒号和类型组成。用正则表达式可描述为：

$$L(\langle \text{形参列表} \rangle) = \epsilon \mid P(, P)^*$$

其中 $P = [\text{mut}] \text{ ID} : \text{Type}$ 。合法的形参列表举例如下：

```
1 fn f()                // 空
2 fn f(a: i32)          // 单参数
3 fn f(mut a: i32, b: i32) // 多参数
4 fn f(a: i32, mut b: i32, c: i32) // 任意长度
```

形式语言分析。从形式语言的角度，该文法虽然是上下文无关文法（2 型），但其递归结构仅用于生成逗号分隔的列表，等价于正则表达式 $\epsilon \mid P(, P)^*$ ，因此 $\langle \text{形参列表} \rangle$ 定义的语言实际上是一个**正则语言**（3 型）。这是因为递归仅出现在产生式的最右端（右递归），没有嵌套结构。

LL(1) 分析。该文法满足 LL(1) 条件：对于 $\langle \text{形参列表} \rangle$ 的三条候选产生式，可以通过查看第一个输入符号做出唯一选择：

- 若当前符号为 `)`，选择 ϵ （空形参列表）
- 若当前符号为 `mut` 或 `Identifier`，选择包含 $\langle \text{形参} \rangle$ 的产生式
- 进入 $\langle \text{形参} \rangle$ 后，由后续是否出现 `,`，决定是否继续递归

因此，该文法可以被递归下降分析器直接处理，无需回溯。

8.2 思考题二：9.1 元组比 8.1 数组多了几条产生式，为什么？

数量对比。

规则 8.1 数组类型新增 1 条产生式：

$$\langle \text{类型} \rangle \rightarrow [\langle \text{类型} \rangle ; \text{NUM}] \quad (105)$$

规则 9.1 元组类型新增 3 条产生式：

$$\langle \text{类型} \rangle \rightarrow (\langle \text{元组类型内部} \rangle) \quad (106)$$

$$\langle \text{元组类型内部} \rangle \rightarrow \epsilon \mid \langle \text{类型} \rangle , \langle \text{类型列表} \rangle \quad (107)$$

$$\langle \text{类型列表} \rangle \rightarrow \epsilon \mid \langle \text{类型} \rangle \mid \langle \text{类型} \rangle , \langle \text{类型列表} \rangle \quad (108)$$

因此，元组类型比数组类型多 2 条产生式（3 vs 1）。

原因分析。

1. **数组元素类型单一（同构）**：数组 $[T; N]$ 中所有元素共享同一类型 T ，数量由常量 N 确定。因此只需一条产生式，将“元素类型”和“长度”两个信息固定在方括号内即可完整描述。
2. **元组元素类型各异（异构）**：元组 $(T_1, T_2, \dots, T_n)$ 中每个位置可以是不同的类型，必须逐一枚举每个位置的独立类型。这需要：
 - 一条产生式定义元组类型的括号结构
 - 一条产生式处理元组内部（含空元组即单元类型 $()$ ）
 - 一条产生式递归处理类型列表
3. **空元组的特殊处理**：元组允许空形式 $()$ （单元类型），需要在 $\langle \text{元组类型内部} \rangle$ 中提供 ϵ 候选。而数组即使长度为 0 也写作 $[T; 0]$ ，结构固定，无需特殊处理。
4. **本质差异**：数组类型的描述是“一个类型 $\times$ 一个数量”的固定模式；元组类型的描述是“任意多个可能不同的类型的序列”，属于列表结构，需要递归产生式来处理不定长的类型枚举。

简言之，数组的**同构性**使其文法天然简洁（一条产生式足够），而元组的**异构性**要求更复杂的文法来逐一描述每个位置的类型（需要三条产生式）。

8.3 部分羁绊分析

作业最后指出了若干条文法规则之间的**羁绊**关系——这些约束超越了纯粹的语法结构，属于**语义层面**的要求。当前的词法分析和语法分析器仅完成前端编译器的语法部分，尚未实现语义分析，因此以下约束在现有系统中不做检测。本节分析每条羁绊的含义及其在语义分析阶段应有的处理方式。

8.3.1 羁绊一：赋值语句 + 类型规则——类型不匹配应报错

规则 2.2 定义了赋值语句 $\langle \text{赋值语句} \rangle \rightarrow \langle \text{左值} \rangle = \langle \text{表达式} \rangle ;$ ，而规则 8.1 等拓展了 $\langle \text{类型} \rangle$ ，使其不再局限于 `i32`，还包含数组类型 $[T;N]$ 、引用类型 $\&T$ 等。

语义约束：当变量声明时指定了类型 T ，后续赋值的表达式类型也应为 T （或可隐式转换为 T ）。例如：

```
1 let a: i32 = 42;           // OK: i32 <- i32
2 let b: [i32; 3] = [1, 2, 3]; // OK: [i32;3] <- [i32;3]
3 let c: i32 = [1, 2, 3]; // ERROR: i32 <- [i32;3]
```

实现思路：需要在语法分析之后引入**类型检查**（Type Checking）阶段。为每个表达式节点标注类型，沿语法树自底向上传播类型信息，在赋值节点处比较左值类型与右值类型是否一致。

8.3.2 羁绊二：for 循环 + 数组——仅数组为可迭代结构

规则 5.2 定义了 for 循环： $\langle \text{for 语句} \rangle \rightarrow \text{for } \langle \text{变量声明} \rangle \text{ in } \langle \text{可迭代结构} \rangle \langle \text{语句块} \rangle$ 。规则 8.2 拓展了 $\langle \text{可迭代结构} \rangle$ ，使其可以是数组。

语义约束：for-in 循环的迭代目标必须是可迭代结构。在当前文法中，可迭代结构包括范围表达式 `start..end` 和数组。其他类型的变量（如 `i32`）不可迭代，应报错。例如：

```
1 for mut i in 0..10 { }      // OK: 范围是可迭代的
2 let arr: [i32; 3] = [1, 2, 3];
3 for mut x in arr { }       // OK: 数组是可迭代的
4 let a: i32 = 42;
5 for mut x in a { }         // ERROR: i32 不可迭代
```

实现思路：在语义分析阶段，检查 for-in 循环中 $\langle \text{可迭代结构} \rangle$ 的类型。若为范围类型或数组类型则合法，否则报错。这需要维护一个**符号表**（Symbol Table），记录每个变量的类型信息。

8.3.3 羁绊三：赋值语句 + 变量不可变属性——对不可变变量赋值应报错

规则 6.1 拓展了 $\langle \text{变量属性} \rangle \rightarrow \epsilon$ ，即变量可以声明为不可变（不加 `mut`）。规则 2.2 和循环规则（5.2 等）中可能存在对不可变变量的赋值。

语义约束：不可变变量只能被声明和读取，不能被重新赋值。例如：

```

1 let a: i32 = 1;
2 a = 2;                                // ERROR: a 不可变，不能赋值
3
4 let mut b: i32 = 1;
5 b = 2;                                // OK: b 是可变变量
6
7 for mut i in 0..10 {
8     i = 5;                             // OK: i 声明为 mut
9 }
```

实现思路：在符号表中为每个变量记录其**可变性**（mutability）属性。在遇到赋值语句时，查询左值对应的变量是否为 `mut`，若不是则报错。

总结。上述三条羁绊均属于语义约束，需要引入**符号表**和**类型系统**才能在编译阶段检测。当前实现聚焦于词法分析和语法分析，语义分析作为后续改进方向。完整的编译器应在语法树构建完成后，遍历语法树进行语义检查（类型检查、可变性检查等），确保程序的语义正确性。

8.4 LR(1) 冲突分析

作业鼓励思考文法中是否存在 LR(1) 无法处理的情况。本节分析类 Rust 文法中的潜在冲突。

8.4.1 经典悬空 else 问题

在许多语言中，if-else 语句存在悬空 else（dangling else）歧义：

```

1 if a > 0 then if b > 0 then x = 1 else x = 2
2 // else 匹配内层 if 还是外层 if?
```

然而，本项目的文法不存在悬空 else 问题。因为规则 4.1 使用 $\langle \text{语句块} \rangle$ （即 $\{ \dots \}$ ）作为 if 和 else 的体，而非 $\langle \text{语句} \rangle$ 。花括号明确界定了每个块的范围，消除了歧义：

```

1 if a > 0 {
2     if b > 0 { ... }
```

```

3 } else {
4     ...
5 }
6 // else 始终匹配外层 if, 因为内层 if 已被 {} 包围

```

8.4.2 规则 7.0 中的移进-归约冲突

规则 7.0 定义了函数表达式语句串：

$$\langle \text{函数表达式语句串} \rangle \rightarrow \langle \text{表达式} \rangle \mid \langle \text{语句} \rangle \langle \text{函数表达式语句串} \rangle \quad (109)$$

而 $\langle \text{语句} \rangle$ 的候选包含 $\langle \text{表达式} \rangle$ ；（规则 3.1 的表达式语句）。因此，当分析器在块内解析了一个 $\langle \text{表达式} \rangle$ 后，面对下一个符号 $;$ 时：

- **归约 (Reduce)**：将 $\langle \text{表达式} \rangle$ 直接归约为 $\langle \text{函数表达式语句串} \rangle$ ，将其视为**尾表达式**（block 的返回值）
- **移进 (Shift)**：移进 $;$ ，与 $\langle \text{表达式} \rangle$ 一起归约为 $\langle \text{语句} \rangle$ ，然后继续解析后续内容

这构成了一个经典的**移进-归约冲突**（Shift-Reduce Conflict）。例如以下代码：

```

1 let mut z = {
2     let mut t = x * x + x;
3     t = t + x * y;
4     t           // <-- 此处是尾表达式还是表达式语句？
5 };

```

在解析到 t 之后，若下一个符号是 $;$ ，分析器不知道 t 是尾表达式（使整个块返回 t 的值）还是需要移进 $;$ 构成表达式语句 $t;$ 。

产生原因：产生式 $\langle \text{函数表达式语句串} \rangle \rightarrow \langle \text{表达式} \rangle$ 和 $\langle \text{语句} \rangle \rightarrow \langle \text{表达式} \rangle$ ；共享了前缀 $\langle \text{表达式} \rangle$ ，且区分符号 $;$ 出现在 $\langle \text{表达式} \rangle$ 之后而非之前，因此仅凭一个向前查看符号无法在归约点做出决策。

解决方案——改写产生式。可以将函数表达式语句串改写为**显式分离**语句部分和尾表达式部分：

$$\langle \text{函数表达式语句块} \rangle \rightarrow \{ \langle \text{语句列表} \rangle [\langle \text{表达式} \rangle] \} \quad (110)$$

$$\langle \text{语句列表} \rangle \rightarrow \epsilon \mid \langle \text{语句} \rangle \langle \text{语句列表} \rangle \quad (111)$$

改写后, $\langle \text{语句列表} \rangle$ 中只包含以 $;$ 结尾的语句, 而 $\langle \text{表达式} \rangle$ 作为可选的尾表达式出现在语句列表之后。决策规则清晰:

- 若当前符号为 $\}$, 且之前有语句, 则语句列表结束, 无尾表达式
- 若当前符号为表达式的起始 (ID 、 NUM 、 $($ 等), 解析 $\langle \text{表达式} \rangle$, 然后检查下一个符号:
 - 若为 $;$, 则这是一个表达式语句, 归入 $\langle \text{语句列表} \rangle$
 - 若为 $\}$, 则这是一个尾表达式

这种改写完全消除了移进-归约冲突, 且与 Rust 语言的实际语义一致: 块内先执行语句, 最后一个不带分号的表达式作为块的返回值。

为什么原始文法难以在 $\text{LR}(1)$ 中处理。根本原因在于 $\langle \text{函数表达式语句串} \rangle$ 的两条候选产生式以同一非终结符 $\langle \text{表达式} \rangle$ 开头, 而区分信息 ($;$ vs $\}$) 出现在 $\langle \text{表达式} \rangle$ 之后。 $\text{LR}(1)$ 的向前查看符号位于归约点, 此时 $\langle \text{表达式} \rangle$ 已被归约, 但 $;$ 尚未被消耗——分析器必须在“将已归约的 $\langle \text{表达式} \rangle$ 作为尾表达式”和“继续移进 $;$ 构成语句”之间做选择。改写后的文法通过引入 $\langle \text{语句列表} \rangle$ 将判断推迟到消耗 $;$ 之后, 从而解决了冲突。

8.4.3 其他潜在冲突分析

元组表达式 vs 括号表达式 (规则 3.1、9.2): 规则 3.1 中 $\langle \text{因子} \rangle \rightarrow (\langle \text{表达式} \rangle)$ 与规则 9.2 中 $\langle \text{可取元素} \rangle \rightarrow (\langle \text{元组赋值内部} \rangle)$ 均以 $($ 开头。但在 $\text{LR}(1)$ 中, 解析完第一个表达式后, 下一个符号是 $)$ 还是 $,$ 可以明确区分两者: $)$ 对应括号表达式, $,$ 对应元组表达式。因此不构成 $\text{LR}(1)$ 冲突。

乘法 vs 解引用 (规则 3.4、6.4): 运算符 $*$ 同时出现在乘法表达式 $\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle * \langle \text{因子} \rangle$ 和解引用表达式 $\langle \text{左值} \rangle \rightarrow * \langle \text{左值} \rangle$ 中。但在 $\text{LR}(1)$ 中, 上下文可以明确区分: 若 $*$ 出现在二元运算符位置 (前一个符号是因子/表达式), 则为乘法; 若出现在一元位置 (前一个符号是语句开头或另一个运算符), 则为解引用。不构成 $\text{LR}(1)$ 冲突。

数组下标 vs 数组字面量 (规则 8.2、8.3): $[$ 既可以开始数组字面量 $[\langle \text{数组元素列表} \rangle]$, 也可以开始数组下标 $\langle \text{可取元素} \rangle [\langle \text{表达式} \rangle]$ 。但前者出现在因子 (表达式起始) 位置, 后者出现在已有可取元素之后, 上下文不同。不构成 $\text{LR}(1)$ 冲突。

9 总结与反思

9.1 项目成果总结

本项目成功实现了类 Rust 语言的前端编译器，主要成果包括：

1. **词法分析器**：基于 Flex 实现，能够正确识别关键字、运算符、界符、常量、标识符、注释等词法单元
2. **语法分析器**：采用递归下降分析法，能够正确解析函数定义、控制流语句、表达式等语法结构
3. **语法树生成**：实现了清晰的语法树数据结构和输出格式
4. **可视化界面**：基于 ImGui 实现了直观的 GUI 展示
5. **跨平台支持**：支持 Linux 和 Windows 双平台编译运行

9.2 遇到的问题与解决方案

在项目开发过程中，我们遇到了以下主要问题：

1. 正则表达式优先级问题

问题：关键字和标识符的正则表达式可能冲突，如 `let` 可能被识别为标识符。

解决方案：在 Flex 规则中将关键字规则置于标识符规则之前，Flex 按规则顺序优先匹配。

2. 左递归问题

问题：表达式文法存在左递归，无法直接用于递归下降分析。

解决方案：采用消除左递归的标准方法，将文法改写为分层结构，并通过循环结构处理连续运算符。

3. 运算符优先级处理

问题：如何正确处理多层次运算符优先级。

解决方案：采用分层解析方法，每层函数对应一个优先级，通过函数调用层次确保高优先级运算符先被处理。

4. 跨平台编译问题

问题：Windows 平台缺少 GLFW 等依赖库。

解决方案：使用 vcpkg 进行跨平台依赖管理，在 Linux 上使用 MinGW-w64 进行交叉编译。

5. 特殊字符处理

问题：字符串常量中的换行符影响输出格式。

解决方案：设计自定义转义规则，将特殊字符转换为可识别的转义序列。

9.3 知识收获与能力提升

通过本次项目实践，我们获得了以下收获：

1. **理论知识深化**：深入理解了词法分析（有限状态自动机）和语法分析（递归下降）的理论基础
2. **工程实践能力**：掌握了 Flex 工具的使用、CMake 构建系统配置、跨平台编译技术
3. **代码设计能力**：学习了语法树数据结构设计、流式接口设计等软件设计方法
4. **团队协作经验**：通过分工合作，提升了项目管理和团队协作能力

9.4 改进方向与展望

项目仍有改进空间，主要包括：

1. **错误恢复机制**：当前语法分析器遇到错误即停止，可改进为错误恢复后继续分析
2. **更多语法特性**：可扩展支持结构体、枚举、泛型等复杂语法
3. **语义分析阶段**：实现类型检查、作用域分析等语义分析功能
4. **代码生成阶段**：生成中间代码或目标代码，完成完整编译流程
5. **性能优化**：优化词法分析的 Token 缓冲机制，提升大文件处理效率

9.5 个人想法与体会

9.5.1 对文法设计的见解

本次作业的文法设计呈现出一种**渐进式拓展**的风格：从最简单的空函数（1.1）开始，逐步添加语句、表达式、控制流、引用、数组、元组等特性。每条新规则都明确标注了所依赖的前置规则和被拓展的规则，构成了一张清晰的**依赖图**。这种设计方法给我们两点启发：

1. **文法可以增量式构建**。在实际的编译器开发中，往往也需要从核心语言开始，逐步添加特性。这种分层次的文法组织方式不仅便于学习理解，也便于分阶段实现和测试。我们的项目正是按此思路推进：先确保必选规则（0.1–5.1）全部通过，再逐步实现扩展规则。
2. **产生式之间存在非平凡的交互**。如”部分羁绊分析”中讨论的，不同规则之间的组合可能引入语义层面的约束（类型检查、可变性检查），这些约束在单独看某条规则时并不明显，只有在组合使用时才会暴露。这说明编译器设计不能只关注单条文法规则，还必须考虑规则之间的全局交互。

此外，我们注意到文法中〈可取元素〉和〈可取引用〉的设计有其巧妙之处：通过将”可以出现在赋值号左侧”的结构统一为一个非终结符，文法自然地描述了 Rust 中**所有权和借用体系**的语法基础。例如，规则 6.2–6.4 通过为〈可取引用〉添加 ‘&’ 和 ‘& mut’ 候选，在语法层面就区分了可变引用和不可变引用。

9.5.2 对实现方案的取舍思考

在项目实现过程中，我们做出了若干设计取舍：

递归下降 vs 工具生成。语法分析器选择了手写递归下降而非使用 Yacc/Bison 等工具生成。手写递归下降的优势在于：

- **错误信息质量高**：可以在每个解析函数中提供精确的上下文信息（如”解析函数参数时期望标识符”），而工具生成的分析器往往只能给出”syntax error”
- **控制力强**：可以灵活处理特殊的语法结构，如 `parseExprOrAssignStmt()` 先解析表达式再判断是否为赋值，避免了将赋值和表达式分为不同入口的复杂性
- **易于调试**：每个解析函数对应一个非终结符，可以单独设置断点、打印日志

代价是需要手动消除左递归和管理运算符优先级。我们采用经典的分层解析方法（每层函数对应一个优先级），虽然代码量稍多，但逻辑清晰、易于维护。

TokenStream 流式接口 vs 数组加载。我们选择将词法分析结果全部加载到内存中的 `vector<Token>`，而非逐 token 喂给语法分析器。这简化了实现（语法分析器可以随时向前查看任意位置），但对于超大文件可能会有内存压力。在实际编译器中（如 GCC、LLVM），通常采用按需读取 + 缓冲的策略，在内存效率和灵活性之间取得平衡。

注释过滤策略。我们选择在词法分析阶段生成注释 Token，但在语法分析之前统一过滤（`remove_comments()`），而非在词法分析阶段直接跳过。这样做的好处是保留了注释信息，可以在 GUI 中展示完整的 Token 流（包括注释），增强了可视化效果。这是一种以少量内存开销换取更好用户体验的权衡。

9.5.3 对 Rust 语言特性的分析

通过实现类 Rust 语言的编译器前端，我们对 Rust 的语法设计有了更深入的理解：

显式类型声明。Rust 要求在函数参数和大部分变量声明中显式标注类型（虽然支持局部类型推导），这与我们文法中 $\langle \text{形参} \rangle \rightarrow \langle \text{变量属性} \rangle \text{ID} : \langle \text{类型} \rangle$ 的设计一致。显式类型使语法分析更简单——不需要复杂的类型推导机制，仅在语法分析阶段就能获取完整的类型信息。

块即表达式。规则 7.0–7.4 揭示了 Rust 的一个核心设计哲学：**几乎一切皆是表达式**。花括号块可以求值，if-else 可以求值，loop 也可以求值。这使得 Rust 不需要三元运算符 `?:`，因为 if-else 本身就是表达式。虽然在本次项目中由于实现复杂度暂未支持这一特性，但这种“表达式导向”的设计理念对我们有深刻启发——它使语言的语法更加统一和正交。

可变性显式标注。Rust 的 `mut` 关键字要求程序员显式声明变量的可变性，默认不可变。这一设计在文法中体现为 $\langle \text{变量属性} \rangle \rightarrow \text{mut} \mid \epsilon$ （规则 0.1 和 6.1）。从编译器的角度看，`mut` 不仅是语法糖，更是一种语义承诺：编译器可以基于可变性信息进行更激进的优化（如不可变变量的值可以被缓存）。同时，它也使得**别名 + 可变性的检查**成为可能，这正是 Rust 借用检查器（Borrow Checker）的核心工作。

运算符的多义性。项目中 `*` 同时表示乘法和解引用，`&` 同时表示按位与和取地址（引用）。在语法层面，这些可以通过上下文区分（二元位置 vs 一元位置），但在语义层面，Rust 进一步利用类型信息消歧——这种“语法相同但语义不同”的设计在编程语言中并不少见，它要求语义分析阶段有足够的类型信息才能做出正确判断。

10 参考文献

1. Rust 程序设计语言中文版. <https://rustwiki.org/zh-CN/book/>
2. 自己动手写编译器. <https://pandolia.net/tinyc/index.html>
3. 北京大学编译实践课程在线文档. <https://pku-minic.github.io/online-doc/>
4. antlr/grammars-v4: Grammars written for ANTLR v4. <https://github.com/antlr/grammars-v4>
5. CLIUtils/CLI11: Command line parser for C++11. <https://github.com/CLIUtils/CLI11>
6. ocornut/imgui: Dear ImGui. <https://github.com/ocornut/imgui>
7. Flex 手册. <https://westes.github.io/flex/manual/>
8. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. 编译原理（第 2 版）. 机械工业出版社.

附录 A 完整语法规则列表

A.1 完整产生式列表

程序结构 (112)

$Program \rightarrow FunctionList$ (113)

$FunctionList \rightarrow Function \mid Function FunctionList$ (114)

$Function \rightarrow FuncHeader Block$ (115)

$FuncHeader \rightarrow \text{fn Identifier (ParamList) } [-> Type]$ (116)

$ParamList \rightarrow \epsilon \mid Param \mid Param , ParamList$ (117)

$Param \rightarrow [\text{mut}] Identifier : Type$ (118)

$Block \rightarrow \{ StmtList \}$ (119)

$StmtList \rightarrow \epsilon \mid Stmt StmtList$ (120)

语句 (122)

$Stmt \rightarrow EmptyStmt \mid LetStmt \mid ReturnStmt \mid IfStmt$ (123)

$\mid WhileStmt \mid ForStmt \mid LoopStmt \mid BreakStmt$ (124)

$\mid ContinueStmt \mid Block \mid AssignStmt \mid ExprStmt$ (125)

$EmptyStmt \rightarrow ;$ (126)

$LetStmt \rightarrow \text{let VarDecl [= Expr] ;}$ (127)

$VarDecl \rightarrow [\text{mut}] Identifier [: Type]$ (128)

$ReturnStmt \rightarrow \text{return [Expr] ;}$ (129)

$AssignStmt \rightarrow LValue = Expr ;$ (130)

$ExprStmt \rightarrow Expr ;$ (131)

$BreakStmt \rightarrow \text{break ;}$ (132)

$ContinueStmt \rightarrow \text{continue ;}$ (133)

控制流 (135)

$IfStmt \rightarrow \text{if Expr Block [ElseClause]}$ (136)

$ElseClause \rightarrow \text{else Block} \mid \text{else IfStmt}$ (137)

$WhileStmt \rightarrow \text{while Expr Block}$ (138)

$ForStmt \rightarrow \text{for } [\text{mut}] Identifier \text{ in Expr Block}$ (139)

$$LoopStmt \rightarrow \text{loop } Block \quad (140)$$

$$(141)$$

$$\text{表达式 (消除左递归后)} \quad (142)$$

$$Expr \rightarrow RangeExpr \quad (143)$$

$$RangeExpr \rightarrow CmpExpr \mid CmpExpr \dots CmpExpr \quad (144)$$

$$CmpExpr \rightarrow AddExpr \mid CmpExpr \ CmpOp \ AddExpr \quad (145)$$

$$CmpOp \rightarrow == \mid != \mid > \mid >= \mid < \mid <= \quad (146)$$

$$AddExpr \rightarrow MulExpr \mid AddExpr \ AddOp \ MulExpr \quad (147)$$

$$AddOp \rightarrow + \mid - \quad (148)$$

$$MulExpr \rightarrow Unary \mid MulExpr \ MulOp \ Unary \quad (149)$$

$$MulOp \rightarrow * \mid / \quad (150)$$

$$Unary \rightarrow \& [\text{mut}] \ Unary \mid * \ Unary \mid Atom \quad (151)$$

$$Atom \rightarrow Constant \mid (Expr) \mid ArrayLit \quad (152)$$

$$\mid Identifier [CallExpr \mid IndexExpr] \quad (153)$$

$$CallExpr \rightarrow (ArgList) \quad (154)$$

$$IndexExpr \rightarrow [Expr] \quad (155)$$

$$ArgList \rightarrow \epsilon \mid Expr \mid Expr \ , \ ArgList \quad (156)$$

$$ArrayLit \rightarrow [ExprList] \quad (157)$$

$$ExprList \rightarrow \epsilon \mid Expr \mid Expr \ , \ ExprList \quad (158)$$

$$(159)$$

$$\text{类型} \quad (160)$$

$$Type \rightarrow \text{i32} \mid Identifier \quad (161)$$

$$\mid [Type ; Constant] \quad (162)$$

$$\mid \& [\text{mut}] \ Type \quad (163)$$

$$\mid (TypeList) \quad (164)$$

$$TypeList \rightarrow \epsilon \mid Type \mid Type \ , \ TypeList \quad (165)$$

附录 B 完整测试输入及输出

B.1 完整词法分析输出

完整词法分析输出共包含约 290 个 Token, 详见项目文件 `parser/testfiles/lexer_output.tsv`。

B.2 完整语法树输出

完整语法树输出包含 3 个函数定义, 共约 480 行, 详见项目文件 `parser/testfiles/parse_tree.txt`。

B.3 运行命令示例

```
1 # Linux
2 ./parser --input input.rs --lexer-output lexer.tsv --parser-output
   tree.txt
3
4 # Windows
5 parser.exe --input input.rs --lexer-output lexer.tsv --parser-output
   tree.txt
6
7 # GUI模式 (Windows)
8 parser.exe --gui --input input.rs
```

Listing 33: 命令行运行示例